



Spring 5 for Beginner

전체 목차

1. Overview Spring 5
2. Travel Club : Spring Framework
3. Travel Club : Spring Boot
4. Travel Club : Spring MVC
5. Travel Club : Spring Data



1. Overview Spring 5

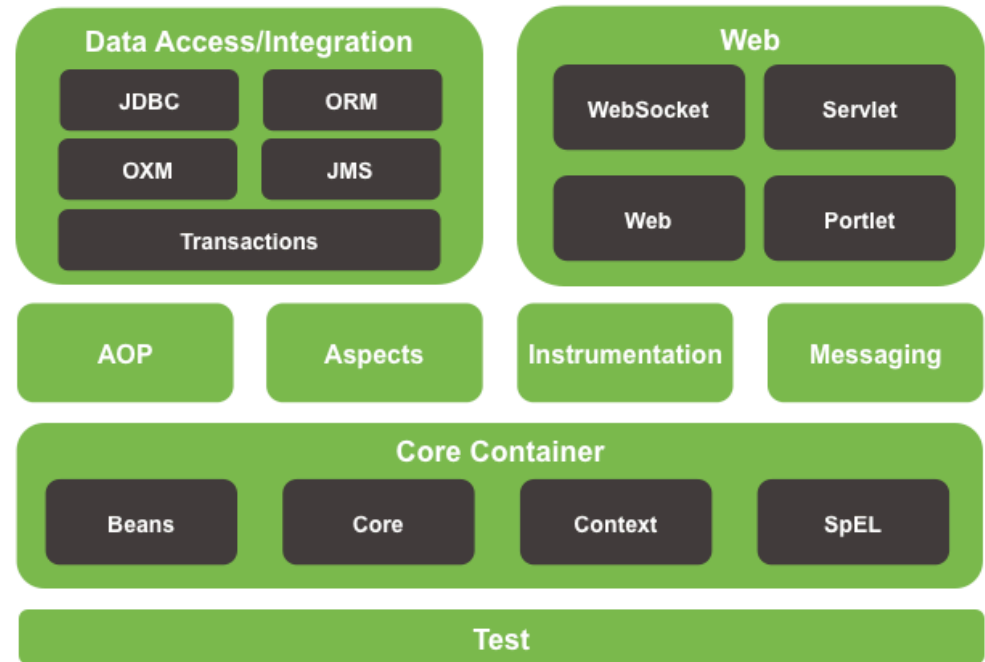
- 1.1 What is Spring?
- 1.2 Spring Framework
- 1.3 Spring MVC
- 1.4 Spring Boot
- 1.5 Spring Data JPA
- 1.6 개발 환경 구성

1.1 What is Spring? (1/2)

- ✓ Spring은 자바 엔터프라이즈 개발을 위한 오픈소스 경량 애플리케이션 프레임워크입니다.
- ✓ 애플리케이션 프레임워크는 애플리케이션 개발의 모든 계층을 지원합니다.
- ✓ 프레임워크가 애플리케이션 수준의 인프라 스트럭처를 제공하므로 개발자는 업무로직 개발에만 전념할 수 있습니다.
- ✓ 스프링 프레임워크는 공통 프로그래밍 모델 및 Configuration 모델을 제공합니다.

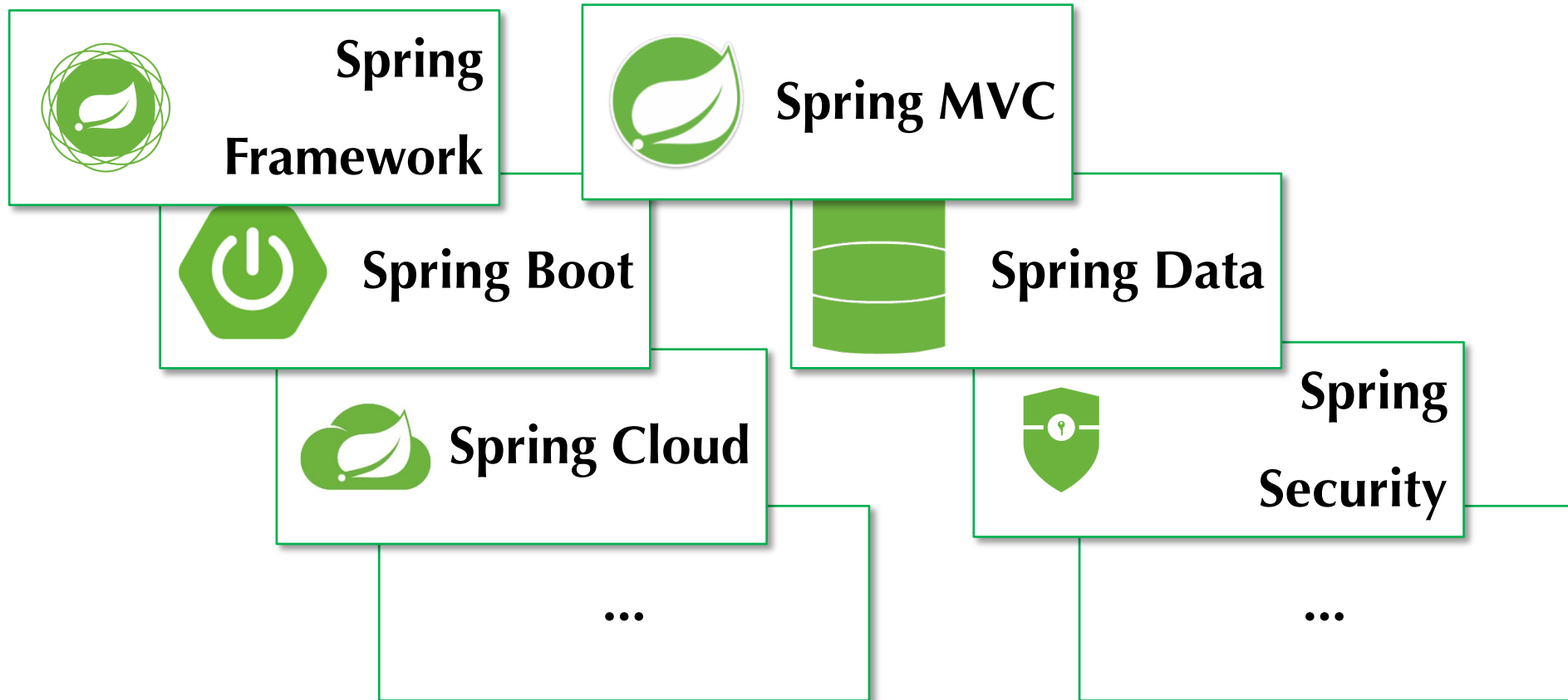


Spring Framework Runtime



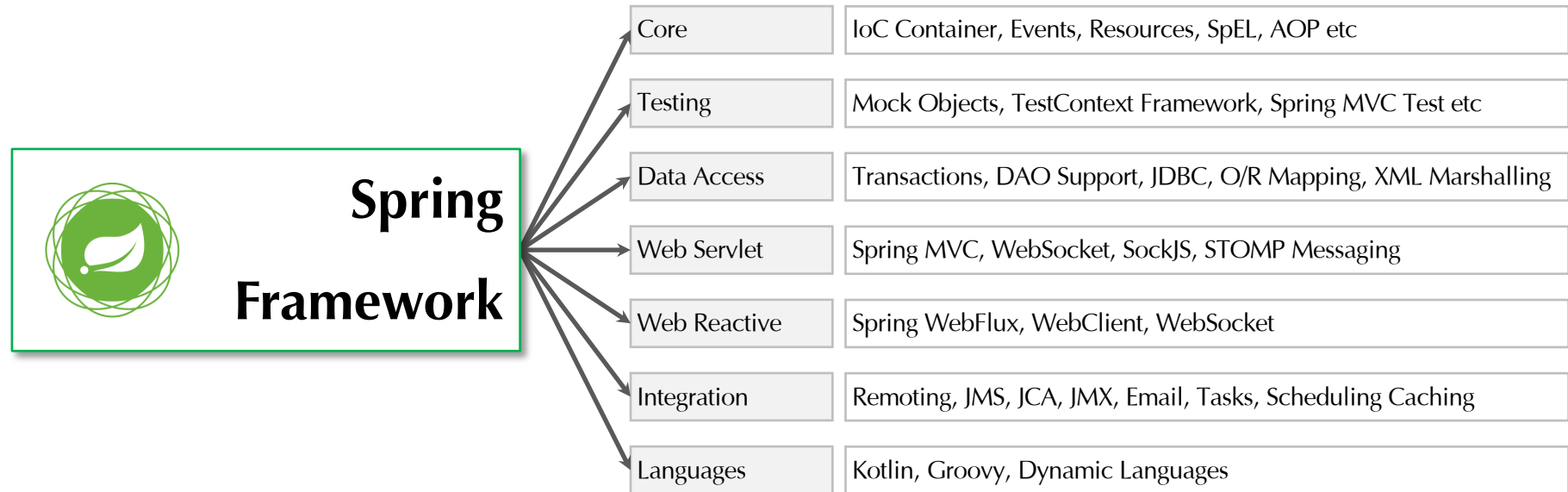
1.1 What is Spring? (2/2)

- ✓ Spring은 다양한 프로젝트로 구분 되며 가장 기본이 되는 프로젝트는 Spring Framework 프로젝트입니다.
- ✓ Spring Framework 프로젝트는 엔터프라이즈 애플리케이션의 핵심 기능을 위한 기반을 제공합니다.
- ✓ 이 밖에도 보안, 클라우드, 빅데이터, 배치 및 배포 등 다양한 프로젝트가 진행중입니다. (<https://spring.io/projects>)
- ✓ Spring Framework, Spring Boot, Spring MVC, Spring Data는 여러 프로젝트 중 핵심이 되는 프로젝트 입니다.



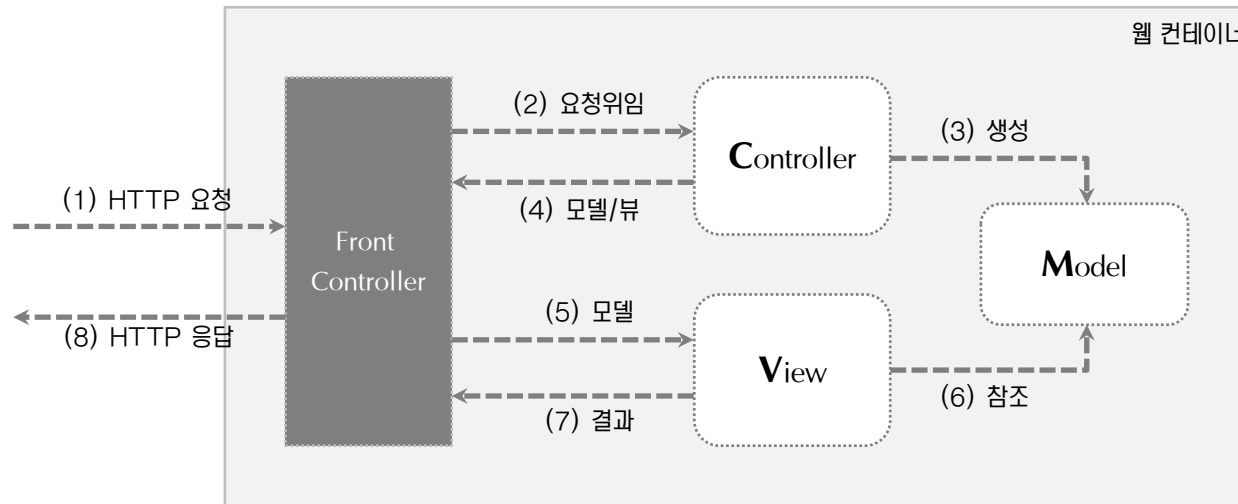
1.2 Spring Framework

- ✓ Spring Framework 프로젝트는 Spring의 핵심(core) 기능들을 담고 있는 프로젝트입니다.
- ✓ 전체 프로젝트를 포함하는 지금 모습과 같은 Spring의 첫 시작은 Spring Framework 입니다.
- ✓ Spring Framework는 엔터프라이즈 애플리케이션 개발을 위해 제공하는 Spring의 핵심 기능들을 담고 있습니다.



1.3 Spring MVC

- ✓ Spring MVC는 MVC 패턴 기반의 웹 프레임워크입니다.
- ✓ Spring MVC는 모든 요청을 받아 각 컨트롤러로 요청을 위임해주는 Front Controller를 사용합니다.
- ✓ 설정보다는 관례(CoC) 중심 프레임워크로서, 보편적인 기능은 기본 제공하고 그 외 설정은 확장 가능합니다.
- ✓ Spring @MVC라고 불리울 만큼 어노테이션을 이용한 편리하고 효율적인 개발을 지원합니다.



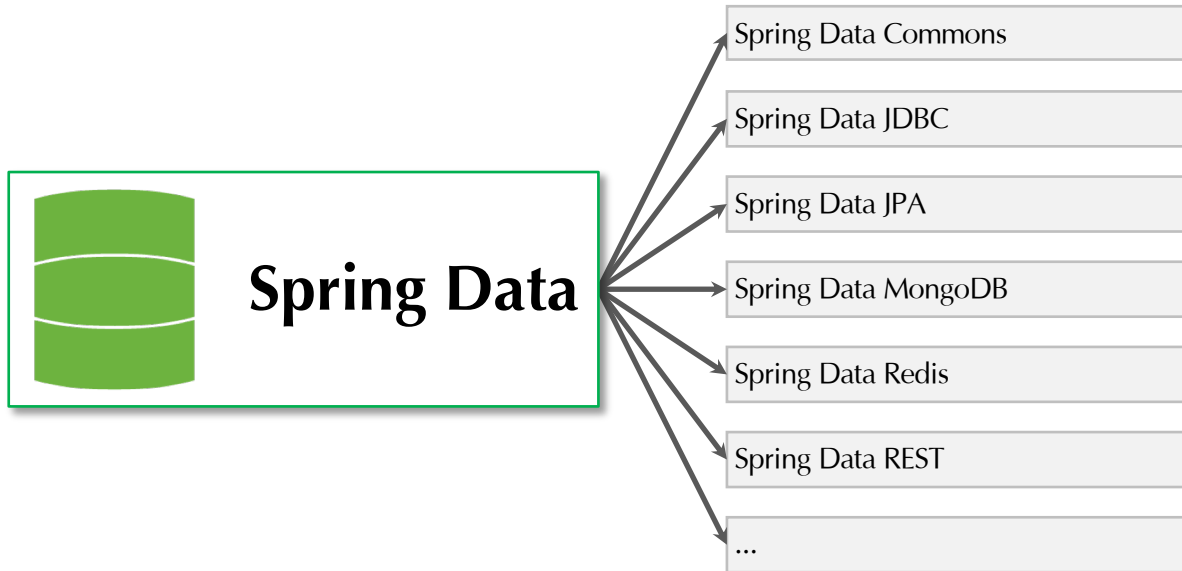
1.4 Spring Boot

- ✓ Spring Boot는 단독 실행할 수 있고, 상용화 수준의, 실행 가능한 스프링 기반 애플리케이션을 쉽게 생성할 수 있게 합니다.
- ✓ Spring Boot를 이용하면 설정의 최소화 할 수 있습니다.
- ✓ 내장된 Tomcat, Jetty, Undertow를 이용해 WAR 배포없이 웹 애플리케이션을 실행할 수 있습니다.
- ✓ 많이 사용하는 라이브러리를 모아놓은 Starter POM 파일을 제공하여 쉽게 라이브러리를 관리할 수 있습니다.



1.5 Spring Data JPA

- ✓ Spring Data 프로젝트는 데이터의 영속성을 위해 사용할 수 있는 모듈의 집합입니다.
- ✓ Spring Data의 다양한 모듈은 관계형 혹은 비 관계형 데이터베이스에 접근 데이터를 관리할 수 있는 방법을 제공합니다.
- ✓ JPA(Java Persistence API)는 ORM(Object-Relation Mapping) 기술 표준입니다.
- ✓ Spring Data JPA 프로젝트는 프로그램에 JPA의 적용을 보다 간결하게 할 수 있도록 합니다.



1.6 개발환경 구성

- ✓ Java Development Kit 8.0 이상
- ✓ REST Client
- ✓ IDE(Integrated Development Environment)





2. Travel Club Project

2.1 Travel Club Project 개요

2.2 UML

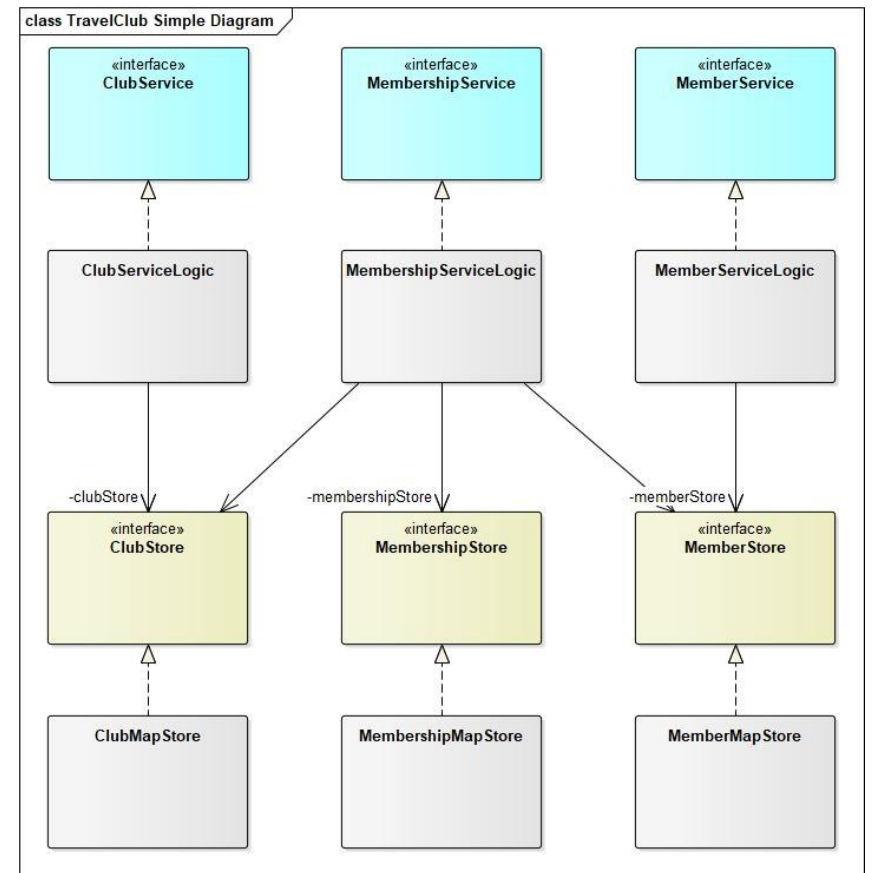
2.3 구현 : Entities, Services, Stores

2.4 이해#01 : Spring core IoC/DI

2.5 이해#02 : Maven 개요

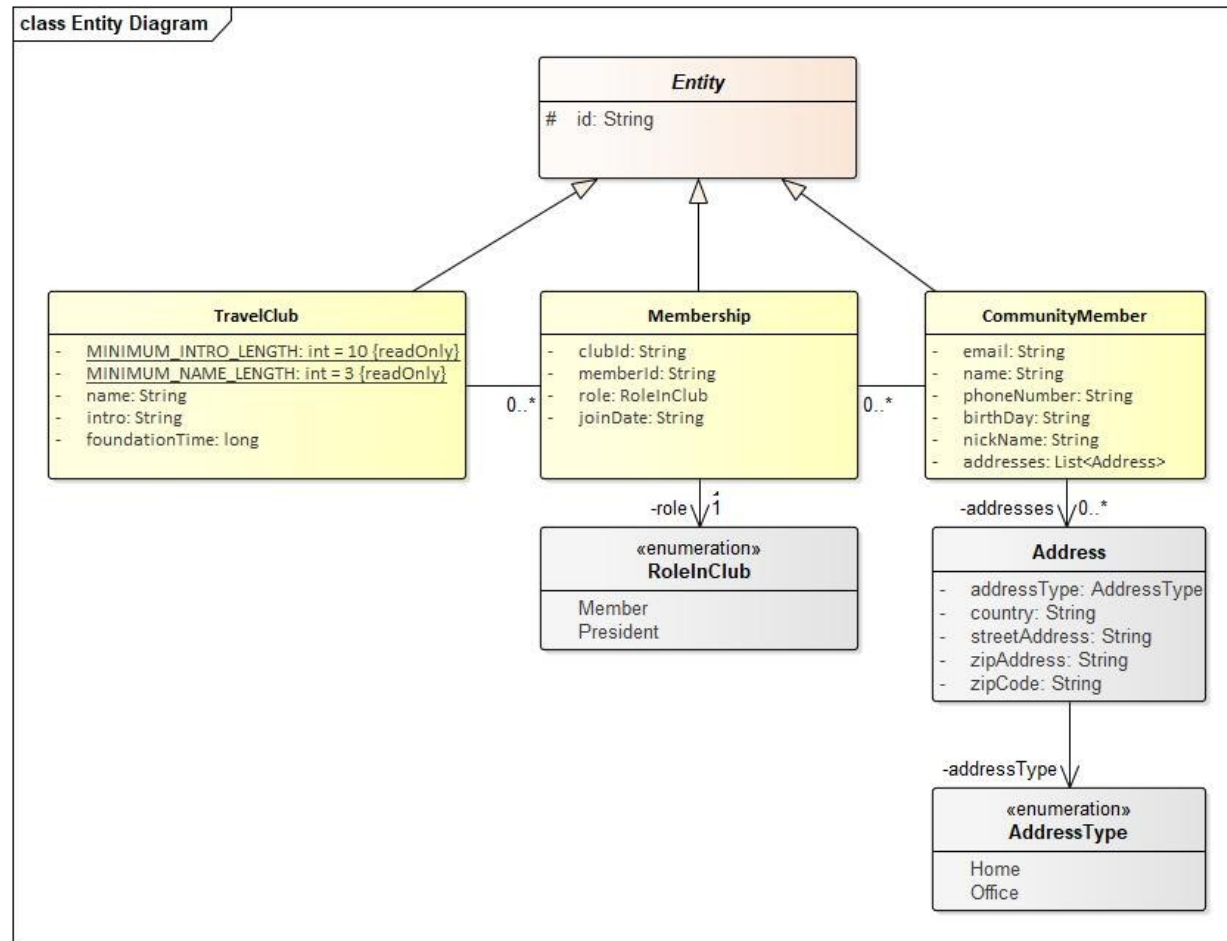
2.1 Travel Club Project 개요

- ✓ Travel Club 프로젝트는 여행 클럽과 클럽 멤버의 관리(클럽 개설과 변경 등)를 위한 어플리케이션 입니다.
- ✓ Travel Club 프로젝트는 크게 4단계의 과정을 통해 개발을 진행합니다.
- ✓ 웹 환경에서 서비스 할 수 있도록 구현하며 사용자 UI(Front-End)는 REST-Client를 이용합니다.



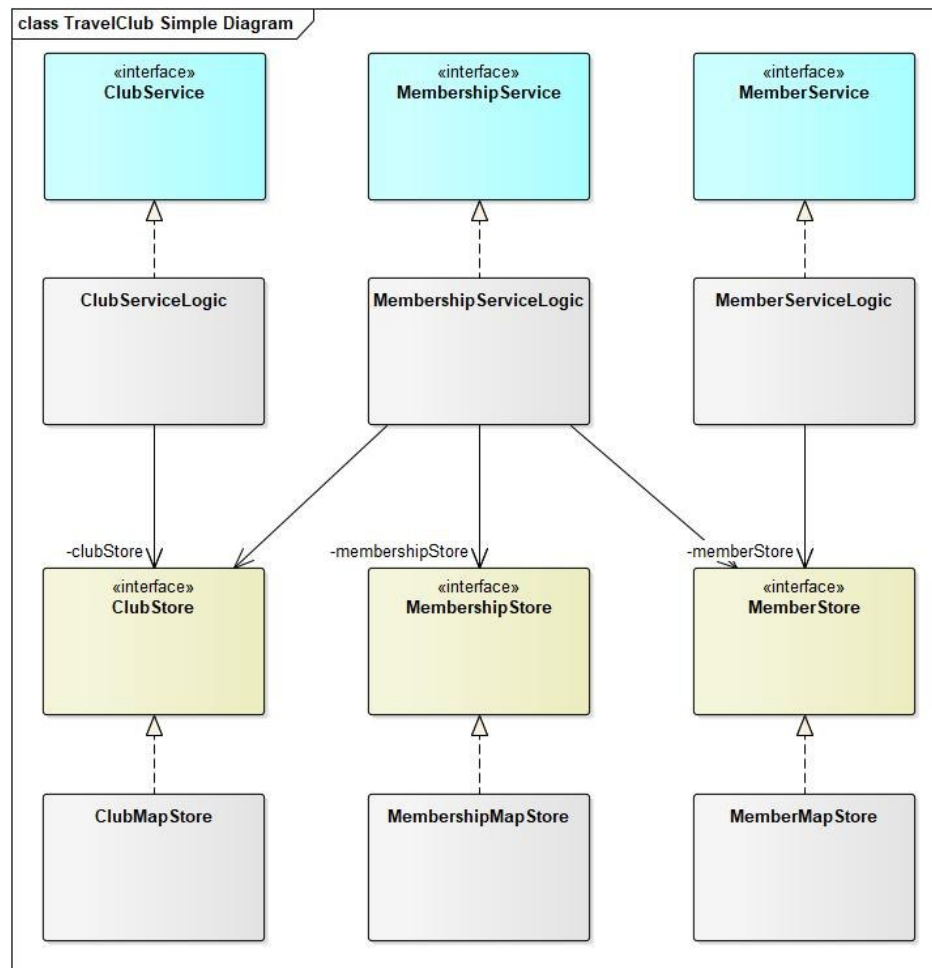
2.2 UML (1/2) – Entities

- ✓ 여행 클럽의 정보는 TravelClub 클래스로 정의하며 클럽명, 소개, 개설일을 정보로 갖습니다.
- ✓ CommunityMember 클래스는 여행 클럽의 멤버에 대한 정보를 담고 있는 클래스입니다.
- ✓ 여행 클럽과 소속 멤버들의 관계는 Membership 클래스를 통해 관리 합니다.



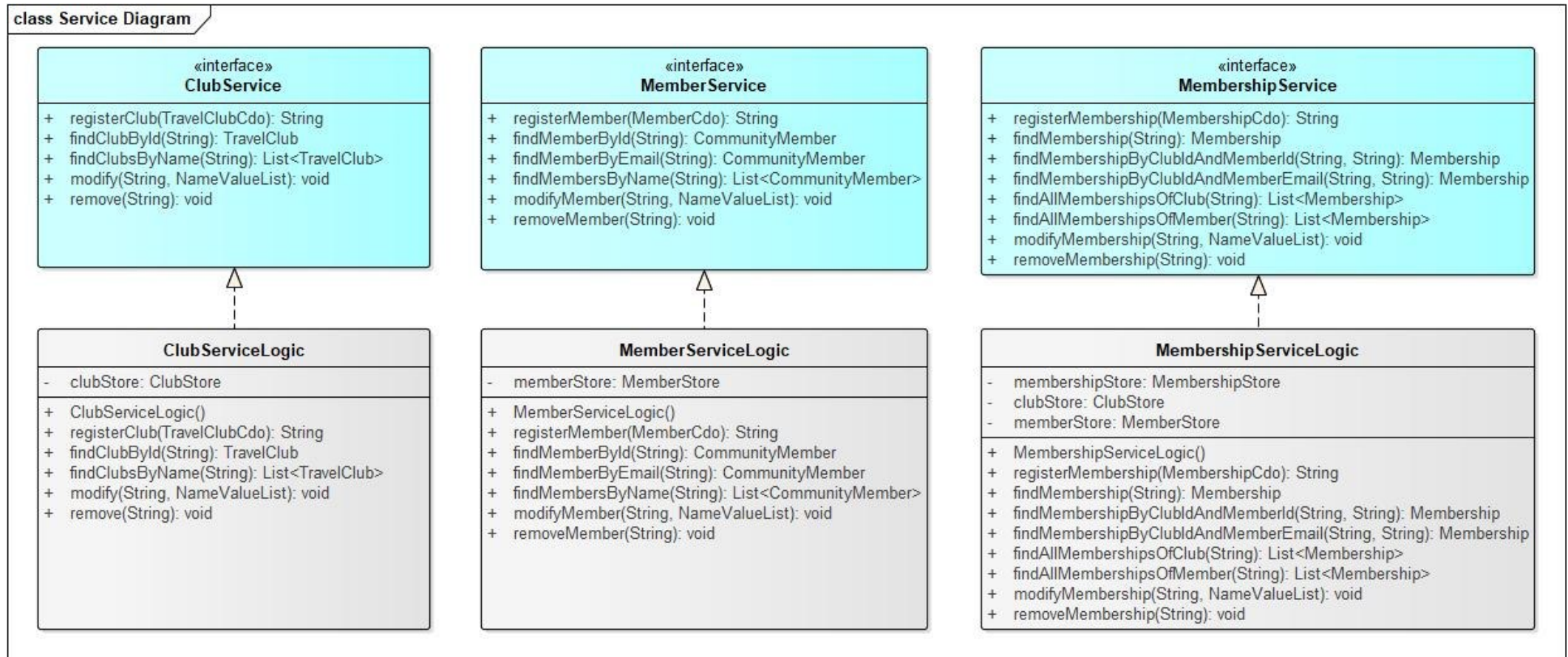
2.2 UML (2/2) – Class Diagram (1/3)

- ✓ 모든 클래스는 그 역할에 따라 구분하고 이를 계층(Layer)으로 나누어 관리 구성합니다.
- ✓ 역할의 구분은 정보(Entity), 처리(Service), 저장(Store)로 계층으로 나눕니다.
- ✓ 처리와 저장에 해당하는 Service와 Store는 인터페이스를 이용해 느슨한 결합(Loose Coupling)을 갖도록 합니다.



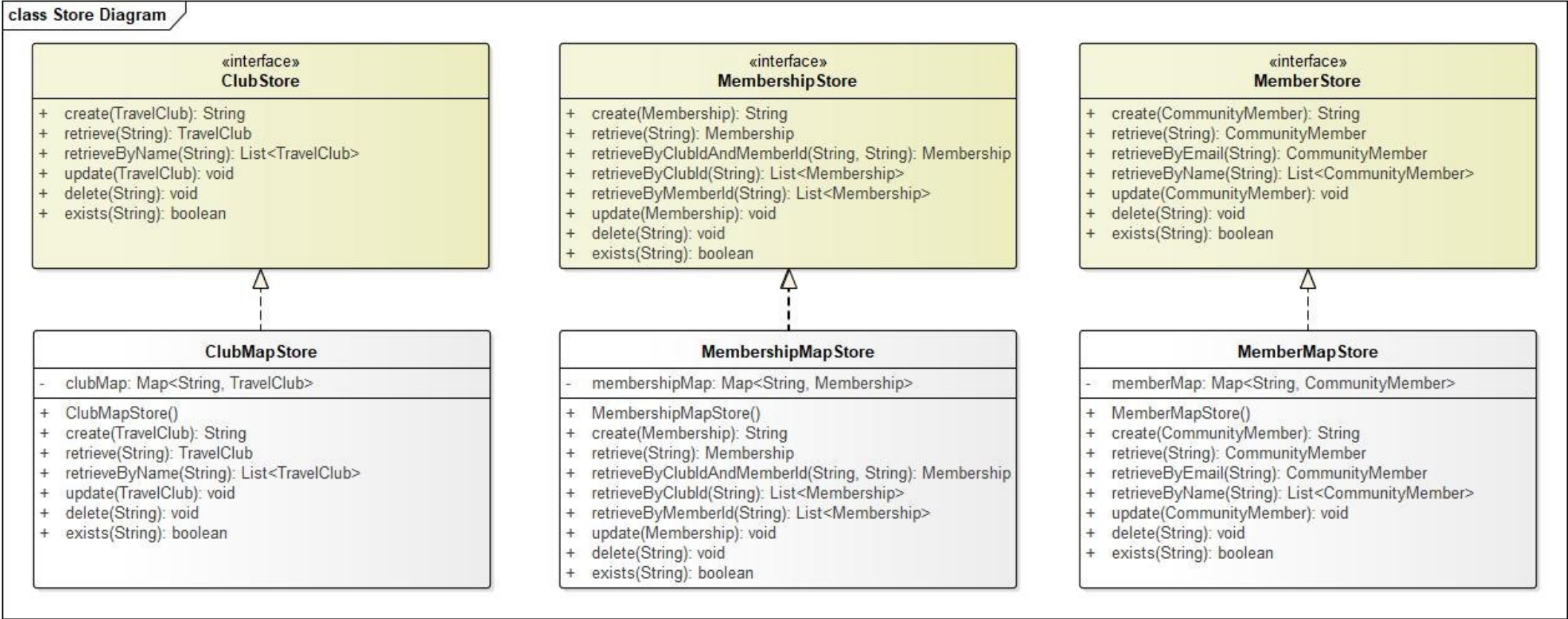
2.2 UML (2/2) – Class Diagram (2/3)

- ✓ 서비스 영역은 클럽, 멤버, 멤버십에 대한 인터페이스와 이를 구현한 구현 클래스로 구성합니다.
- ✓ 각 서비스의 구현체 클래스는 스토어 영역의 인터페이스를 속성으로 참조합니다.
- ✓ **Membership**을 구현한 클래스는 클럽, 멤버, 그리고 멤버십에 대한 스토어 인터페이스의 참조를 갖습니다.



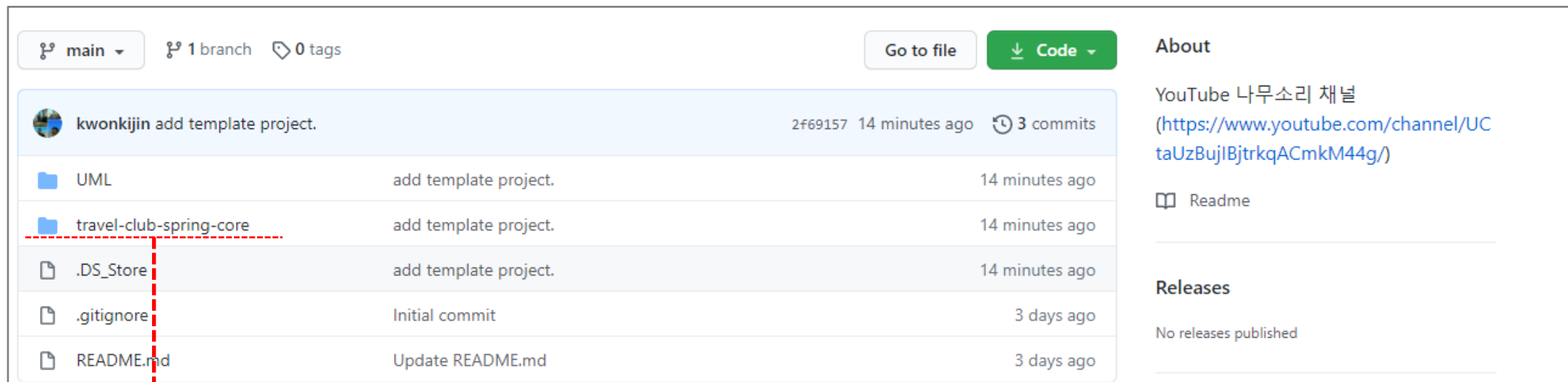
2.2 UML (2/2) – Class Diagram (3/3)

- ✓ 스토어 영역은 클럽, 멤버 그리고 멤버십에 대한 인터페이스와 이를 구현한 구현체로 구성합니다.
- ✓ 스토어 영역의 구현 클래스들은 각 데이터에 대한 저장, 검색, 수정, 삭제 기능을 수행합니다.
- ✓ Spring Data JPA를 적용하는 4단계에는 DB를 통해 데이터를 저장하며 1~3단계에서는 Map을 이용합니다.



2.3 구현

<https://github.com/namoosori/spring5-for-beginner>



main 1 branch 0 tags

Go to file Code

kwonkijin add template project. 2f69157 14 minutes ago 3 commits

UML	add template project.	14 minutes ago
travel-club-spring-core	add template project.	14 minutes ago
.DS_Store	add template project.	14 minutes ago
.gitignore	Initial commit	3 days ago
README.md	Update README.md	3 days ago

About

YouTube 나무소리 채널
(<https://www.youtube.com/channel/UCtaUzBujlBjtrkqACmkM44g/>)

Readme

Releases

No releases published

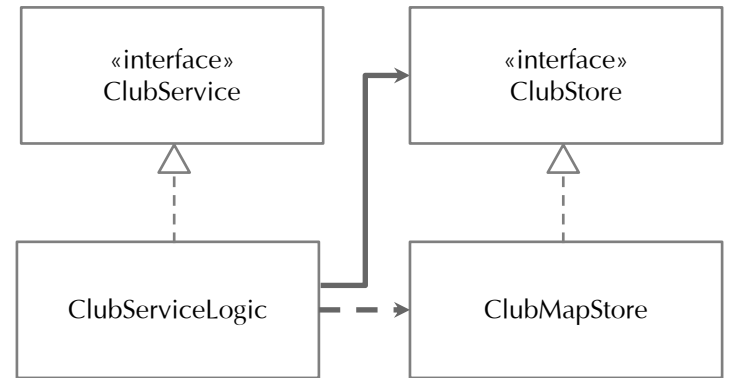
→ Step#01 Travel Club Project(template)

2.4 Spring core IoC/DI (1/15) – 개요 (1/3)

- ✓ 일반적인 객체지향 프로그램에서는 객체를 사용하는 시점에 직접 객체를 생성하여 사용합니다.
- ✓ 객체간의 의존성을 줄이기 위해서 인터페이스와 추상 클래스를 이용합니다.
- ✓ 하지만 소스 코드 상에서 하드-코딩한 객체 생성 구현은 객체 간의 의존성을 강요합니다.
- ✓ 결국, 의도한 것은 아니지만 직접적인 객체 생성으로 인해 객체 간의 결합도가 높아지게 됩니다.

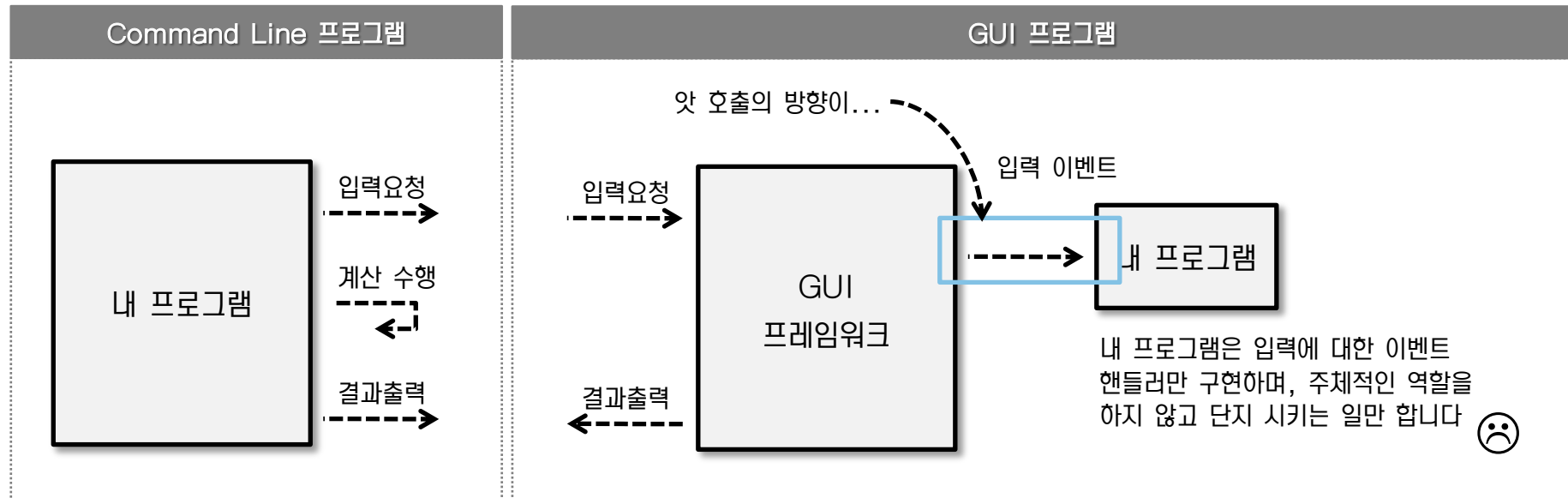
```
public class ClubServiceLogic implements ClubService {  
    //  
    private ClubStore clubStore;  
  
    public ClubServiceLogic() {  
        //  
        this.clubStore = new ClubMapStore();  
    }  
  
    @Override  
    public String registerClub(TravelClubCdo clubCdo) {  
        //  
        TravelClub club = new TravelClub(clubCdo.getName(), clubCdo.getIntro());  
        club.checkValidation();  
        String clubId = clubStore.create(club);  
        return clubId;  
    }  
    ...  
}
```

클래스 다이어그램



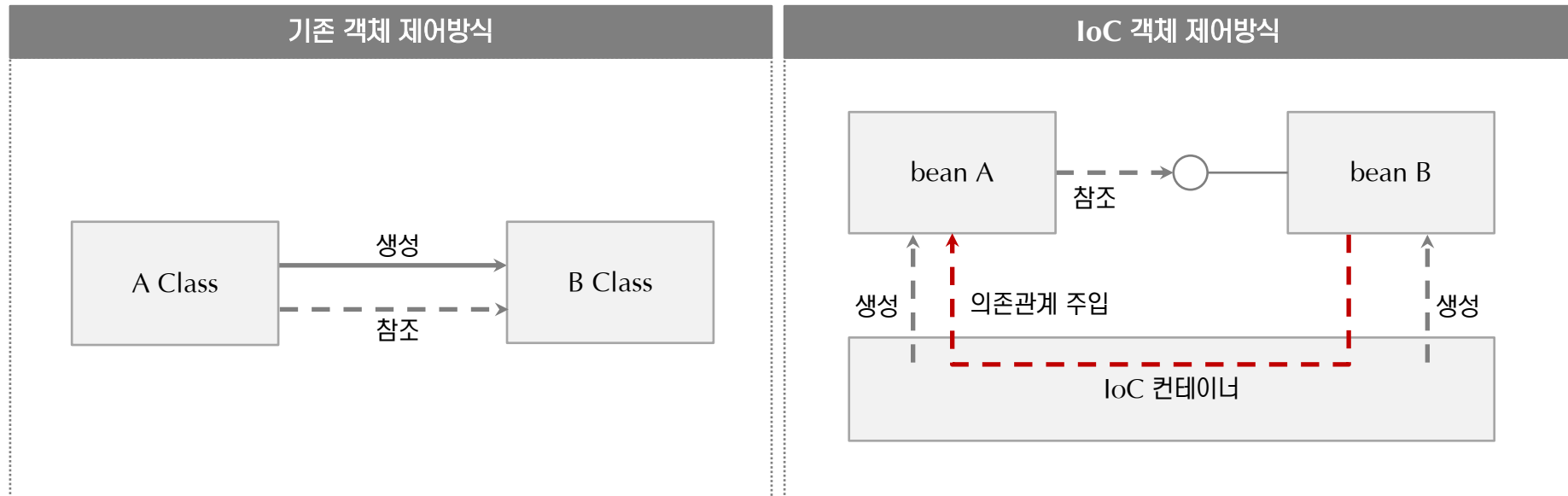
2.4 Spring core IoC/DI (2/15) – 개요 (2/3)

- ✓ IoC(inversion of Control)는 통제 방향의 변경을 의미합니다.
- ✓ 프레임워크가 갖는 일반적인 속성으로 특정 일에 대한 주체를 변경하는 것입니다.
- ✓ 최초의 Command Line 프로그램은 내 프로그램이 모든 것을 통제합니다.
- ✓ 그와 달리, GUI 프로그램은 프레임워크가 통제하며 내 프로그램에서는 이벤트 핸들러만 구현합니다.



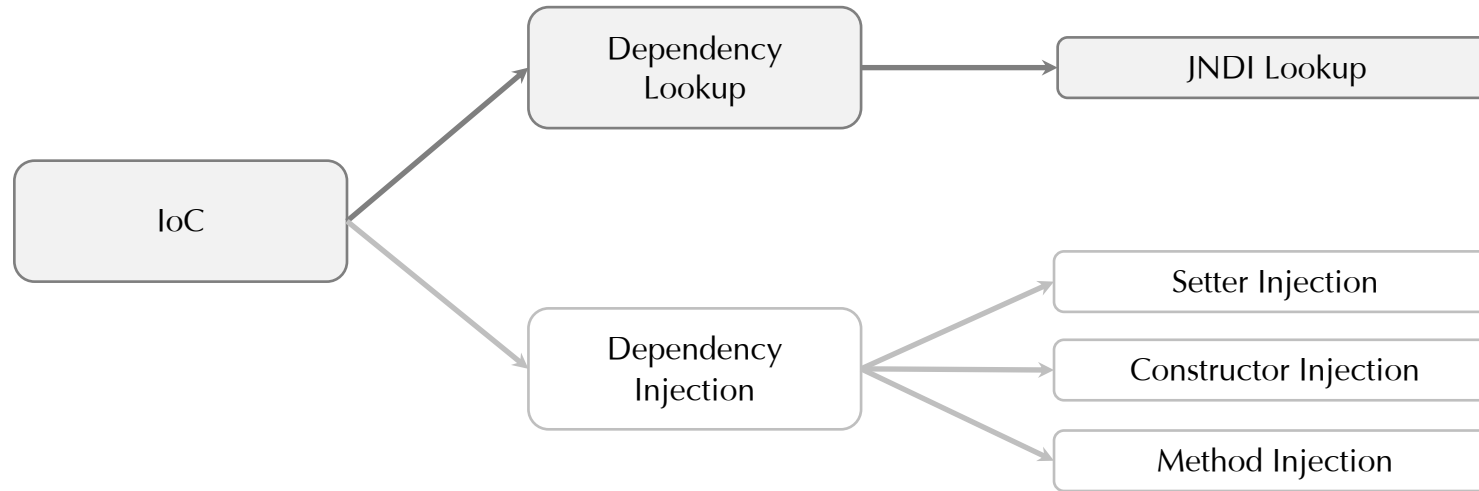
2.4 Spring core IoC/DI (3/15) – 개요 (3/3)

- ✓ 객체 간 결합도가 높으면, 해당 클래스가 변경될 때 결합된 다른 클래스도 같이 수정될 가능성이 있습니다.
- ✓ IoC는 객체 생성 책임을 컨테이너에게 위임하여, 객체간의 결합도를 낮춥니다. (loose coupling)
- ✓ IoC를 통한 객체 제어 방식은 기존 로직에서 객체를 생성하는 로직을 제거합니다.
- ✓ IoC는 구현하는 방법에 따라, Dependency Lookup과, Dependency Injection 방법이 있습니다.



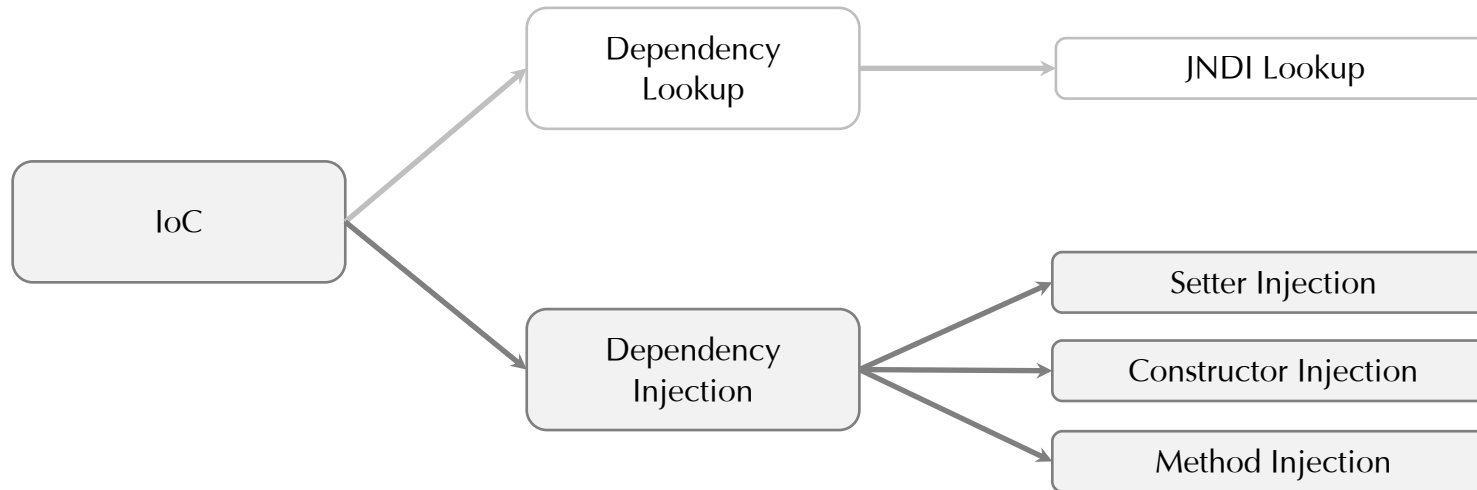
2.4 Spring core IoC/DI (4/15) – Dependency Lookup

- ✓ Dependency Lookup 방식은 EJB나 Spring 에서 JNDI 리소스를 얻는 방식입니다.
- ✓ 컨테이너가 제공하는 Lookup Context 를 통해서 필요한 자원이나 객체를 얻을 수 있습니다.
- ✓ 그러나, 이 방식은 결과적으로 컨테이너 API에 대한 의존성을 높이게 됩니다.
- ✓ 컨테이너와의 의존성을 줄이기 위해서는 DI 방식을 사용합니다.



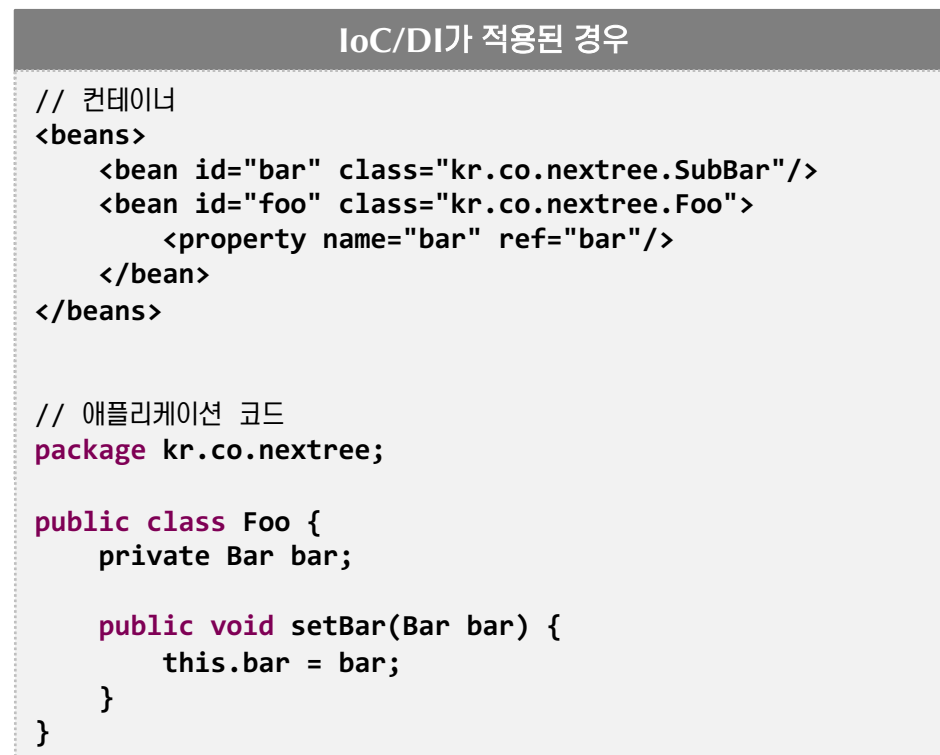
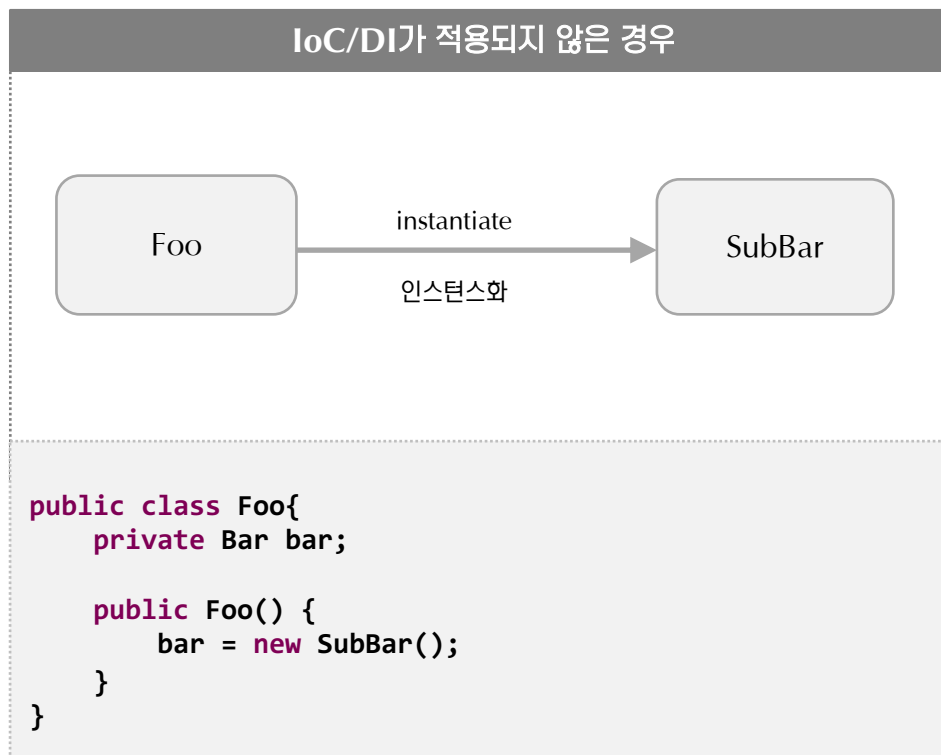
2.4 Spring core IoC/DI (5/15) – Dependency Injection (1/3)

- ✓ 의존 관계 주입 **Dependency Injection**를 줄여서 **DI**라고 합니다.
- ✓ **DI**를 통해, 외부에서 객체를 생성하는 시점에 참조하는 객체에게 의존관계를 제공합니다.
- ✓ 협업 객체의 참조를 어떻게 얻어낼 것인가라는 관점에서 책임성의 역행(**inversion of responsibility**)이라고도 합니다.
- ✓ 객체가 인터페이스만 알고 있으므로, 느슨한 결합(**loose coupling**)이 가능합니다.



2.4 Spring core IoC/DI [6/15] – Dependency Injection (2/3)

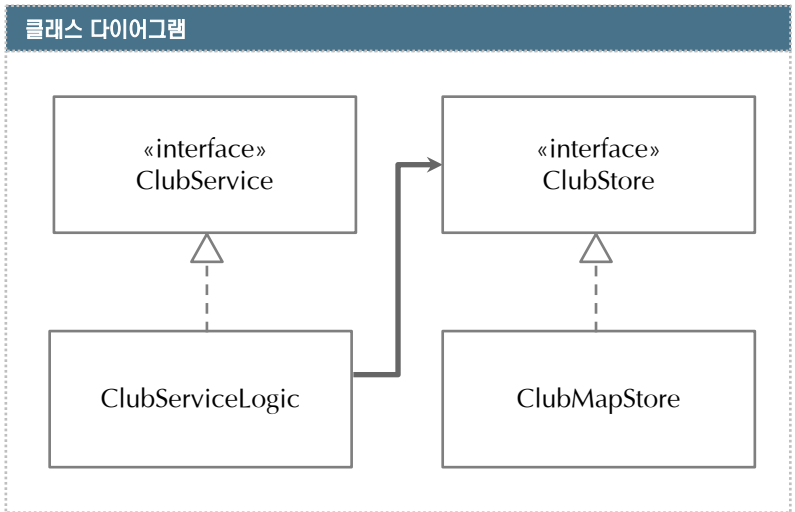
- ✓ DI는 클래스 사이의 의존관계를 빈 설정 정보를 바탕으로 컨테이너가 자동적으로 연결해주는 것을 말합니다.
- ✓ 개발자들은 제어를 담당할 필요 없이 빈 설정 파일에 의존관계가 필요하다는 정보만 추가해주면 됩니다.



2.4 Spring core IoC/DI (7/15) – Dependency Injection (3/3)

- ✓ 사용하는 객체 생성을 IoC 컨테이너에게 위임하여, 객체 생성 로직이 없어졌습니다.
- ✓ 객체 생성에 대한 주도권을 객체를 필요로 하는 곳으로 넘겨주어, 필요할 때 필요한 곳에서 객체를 생성하는 방법입니다.
- ✓ Spring은 Setter Injection, 생성자 Injection, 메소드 Injection 등 세가지 DI 패턴을 모두 지원합니다.
- ✓ Spring은 의존관계를 설정하는 방법으로 XML, 어노테이션, 자바소스를 사용하는 방법을 제공합니다.

```
public class ClubServiceLogic implements ClubService {  
    //  
    private ClubStore clubStore;  
  
    public ClubServiceLogic(ClubStore clubStore) {  
        //  
        this.clubStore = clubStore;  
    }  
  
    @Override  
    public String registerClub(TravelClubCdo clubCdo) {  
        //  
        TravelClub club = new TravelClub(clubCdo.getName(), clubCdo.getIntro());  
        club.checkValidation();  
        String clubId = clubStore.create(club);  
        return clubId;  
    }  
    ...  
}
```



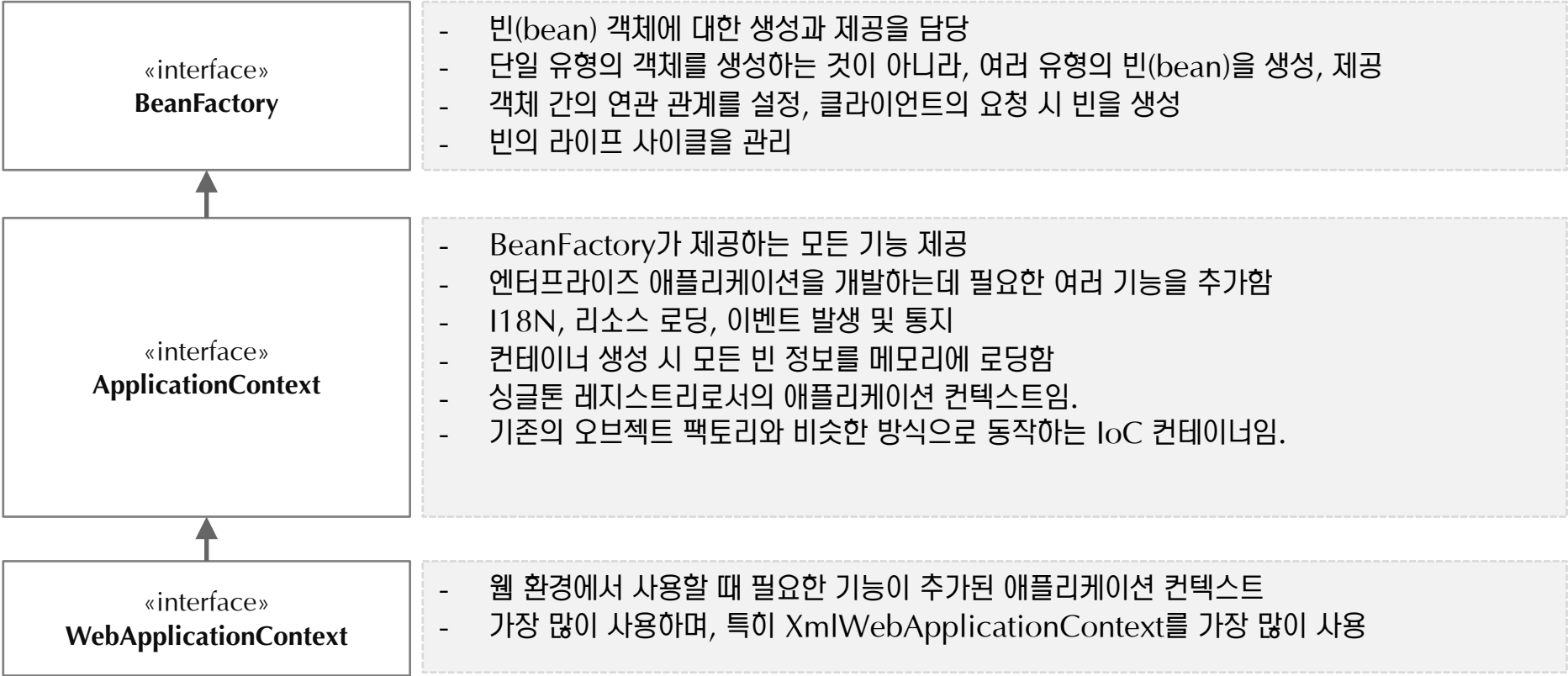
2.4 Spring core IoC/DI (8/15) – Spring IoC 용어

- ✓ 빈(Bean)이란, Spring이 IoC 방식으로 관리하는 객체로, 관리되는 객체(Managed Object)라고도 합니다.
- ✓ IoC 컨테이너는 빈의 생성과 제어의 관점에서 빈 팩토리(Bean Factory)라고도 합니다.
- ✓ 애플리케이션 컨텍스트는 Spring이 제공하는 애플리케이션 지원 기능을 모두 포함하는 의미입니다.
- ✓ Spring 프레임워크는 IoC 컨테이너와 애플리케이션 컨텍스트를 포함한 Spring의 모든 기능을 포괄합니다.

IoC 용어	설명
빈 (bean)	스프링이 IoC 방식으로 관리하는 오브젝트 관리 되는 오브젝트라고도 함. 스프링이 직접 그 생성과 제어를 담당하는 오브젝트만을 빈(bean)이라고 함.
빈 팩토리 (bean factory)	스프링이 IoC를 담당하는 핵심 컨테이너로 빈을 등록, 생성하고, 조회하고 돌려주고, 그 외에 부가적으로 빈을 관리하는 기능을 담당함. 보통은 이 빈 팩토리를 바로 사용하지 않고 이를 확장한 애플리케이션 컨텍스트를 이용합니다.
애플리케이션 컨텍스트 (application context)	빈 팩토리를 확장한 IoC 컨테이너로 빈을 등록하고 관리하는 기본적인 기능은 빈 팩토리와 동일함. 스프링이 제공하는 각종 부가 서비스를 추가로 제공함.
설정정보/설정 메타정보 (configuration metadata)	애플리케이션 컨텍스트 또는 빈 팩토리가 IoC를 적용하기 위해 사용하는 메타정보
스프링 프레임워크	IoC 컨테이너, 애플리케이션 컨텍스트를 포함해서 스프링이 제공하는 모든 기능 칭함.

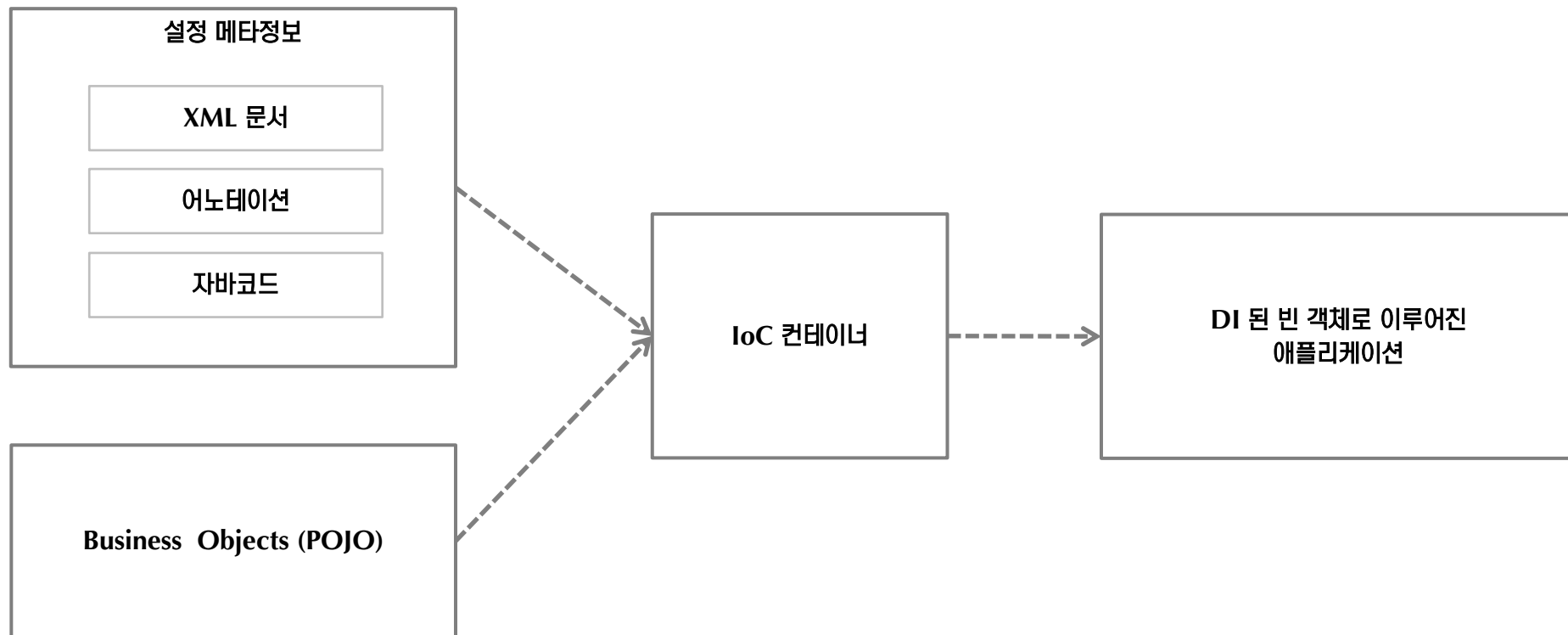
2.4 Spring core IoC/DI (9/15) – IoC 컨테이너

- ✓ 객체의 생성과 관계설정, 사용, 제거 등의 작업을 실제 코드 대신 독립된 컨테이너가 담당합니다.
- ✓ 코드 대신 컨테이너가 객체에 대한 제어권을 갖게 되어 이를 IoC라 하며, Spring 컨테이너를 IoC 컨테이너라고 합니다.
- ✓ Spring에서 IoC를 담당하는 컨테이너는 BeanFactory, ApplicationContext가 있습니다.
- ✓ Spring은 별다른 설정이 없을 경우 컨테이너가 관리하는 Bean 객체를 Singleton으로 생성합니다.



2.4 Spring core IoC/DI (10/15) – 설정 메타 정보

- ✓ 하나의 애플리케이션은 IoC 컨테이너에 의해 POJO 클래스와 설정 메타정보가 결합되어 만들어집니다.
- ✓ 설정 메타정보는 애플리케이션을 구성하는 객체와 객체 사이의 상호 의존성을 포함합니다.
- ✓ Spring은 XML설정 파일과 어노테이션 설정, 자바소스로 메타정보의 설정이 가능합니다.



2.4 Spring core IoC/DI (11/15) – XML 빈(bean) 관리

- ✓ XML을 통해 Spring 빈을 설정할 수 있습니다. XML을 통한 빈 설정은 단순하며 사용하기 쉽습니다.
- ✓ 일반적으로 사용하는 방식으로 빈의 설정 메타 정보를 XML문서에 기술합니다.
- ✓ XML 설정파일에 <bean> 태그를 작성하여 빈을 선언합니다.
- ✓ Spring은 빈을 선언함과 동시에 <property> 태그의 ref 요소를 이용하여 의존관계를 설정합니다.

```
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:context="http://www.springframework.org/schema/context" xsi:schemaLocation="
        http://www.springframework.org/schema/beans https://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context https://www.springframework.org/schema/context/spring-context.xsd">

    <bean id="clubStore"
        class="io.namoo.sori.travelclub.spring.store.mapstore.ClubMapStore">
    </bean>

    <bean id="clubService"
        class="io.namoo.sori.travelclub.spring.service.logic.ClubServiceLogic">
        <constructor-arg ref="clubStore" />
    </bean>
</beans>
```

```
public class ClubServiceLogic implements ClubService {
    //
    private ClubStore clubStore;

    public ClubServiceLogic(ClubStore clubStore) {
        //
        this.clubStore = clubStore;
    }
    ...
}
```

2.4 Spring core IoC/DI (12/15) – Annotation Bean 관리 (1/2)

- ✓ 클래스에 어노테이션을 추가하는 방식으로 Spring 빈을 설정할 수 있습니다.
- ✓ 빈으로 사용할 클래스에 특별한 어노테이션을 부여하면 자동으로 빈을 등록합니다.
- ✓ Spring은 오프젝트 빈 스캐너를 이용한 빈 스캐닝을 통해 자동으로 빈을 등록합니다.
- ✓ 빈 스캐너 기본적으로 빈의 아이디를 클래스 이름(클래스 이름의 첫 글자만 소문자로 바꾼 형태)을 사용합니다.

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context" xsi:schemaLocation="
http://www.springframework.org/schema/beans https://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context https://www.springframework.org/schema/context/spring-context.xsd">

    <context:component-scan base-package="io.namoo.sori.travelclub.spring"/>

</beans>
```

```
@Repository("clubStore")
public class ClubMapStore implements ClubStore {
    //
    private Map<String, TravelClub> clubMap;

    public ClubMapStore() {
        //
        this.clubMap = new LinkedHashMap<>();
    }
    ...
}
```

```
@Service("clubService")
public class ClubServiceLogic implements ClubService {
    //
    private ClubStore clubStore;

    public ClubServiceLogic(ClubStore clubStore) {
        //
        this.clubStore = clubStore;
    }
    ...
}
```

2.4 Spring core IoC/DI (13/15) – Annotation Bean 관리 (2/2)

- ✓ Spring은 ApplicationContext의 BeanDefinition을 통해 빈을 찾습니다.
- ✓ @Repository, @Service, @Controller, @Component 스테레오타입 클래스들을 자동으로 인식합니다.
- ✓ Spring이 관리하는 모든 컴포넌트를 계층 별로 빈의 특성이나 종류에 따라 스테레오타입 어노테이션을 사용합니다.
- ✓ 이를 통해, AOP 포인트컷 표현식을 사용하여 특정 어노테이션을 선언한 클래스만을 선정할 수 있습니다.

스테레오타입 어노테이션		설명
특정 계층에서 사용하는 스테레오타입 어노테이션	@Repository	데이터 액세스 계층의 DAO 또는 Repository 클래스에 사용 DataAccessException 자동변환과 같은 AOP의 적용 대상을 선정하기 위해 사용하기도 함
	@Service	서비스 계층의 클래스에 사용
	@Controller	프레젠테이션 계층의 MVC 컨트롤러에 사용. 스프링 웹 서블릿에 의해 웹 요청을 처리하는 컨트롤러 빈으로 선정
	@Component	위의 계층 구분을 적용하기 어려운 일반적인 경우에 사용

2.4 Spring core IoC/DI (14/15) – Spring Bean Scope

- ✓ Spring 빈의 생성범위에는 singleton, prototype, request, session이 있습니다.
- ✓ Spring 빈은 기본적으로 모든 빈을 singleton 빈(Singleton Bean)으로 생성합니다.
- ✓ 따라서, 컨테이너가 제공하는 빈의 인스턴스는 항상 동일합니다.
- ✓ 컨테이너가 항상 새로운 인스턴스를 반환하게 만들고 싶다면 scope을 prototype으로 설정해야 합니다.

Scope	설명
singleton	스프링 컨테이너 당 하나의 인스턴스 만 생성 (디폴트)
prototype	컨테이너에 빈을 요청할 때 마다 새로운 인스턴스 생성
request	HTTP 요청 별로 새로운 인스턴스를 생성
session	HTTP 세션 별로 새로운 인스턴스를 생성

XML 방식으로 선언한 빈의 Scope 설정

```
<beans xmlns="http://www.springframework.org/schema/beans"
.....>

    <bean id="recipeDao"
          class="com.namoo.yorizori.dao.jdbc.JdbcRecipeDao"
          scope="prototype" />
```

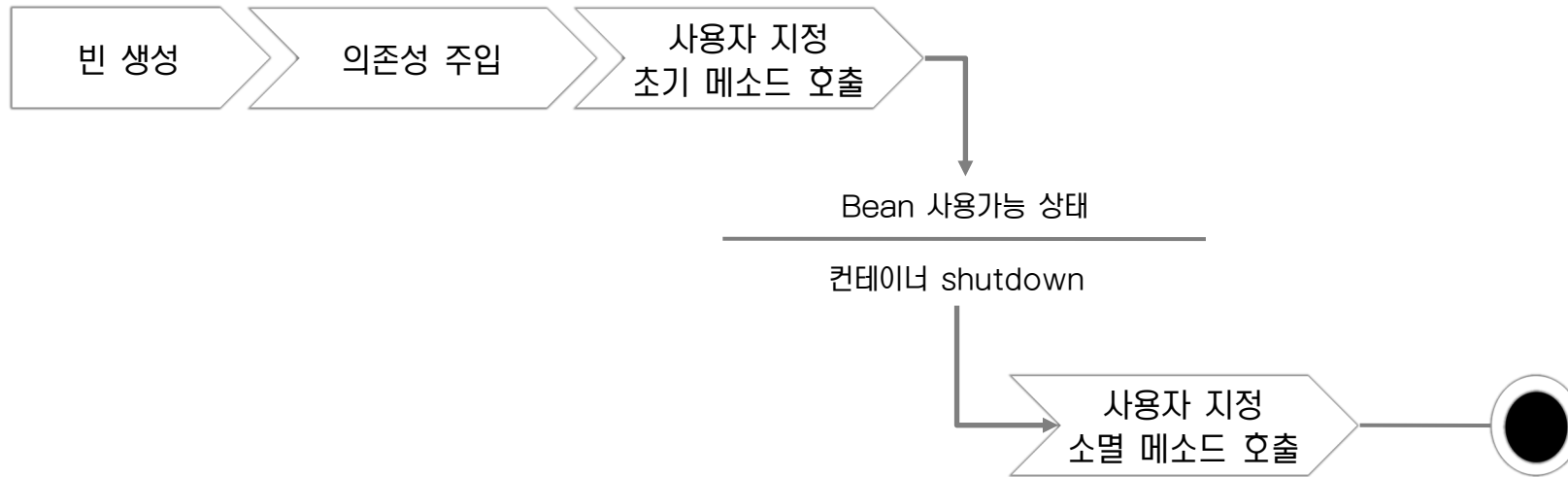
Annotation 방식으로 선언한 빈의 Scope 설정

```
@Repository
@Scope("prototype")
public class JdbcRecipeDao implements RecipeDao {

    @Resource
    private DataSource dataSource;
    ...
}
```

2.4 Spring core IoC/DI (15/15) – Spring Bean 생명주기

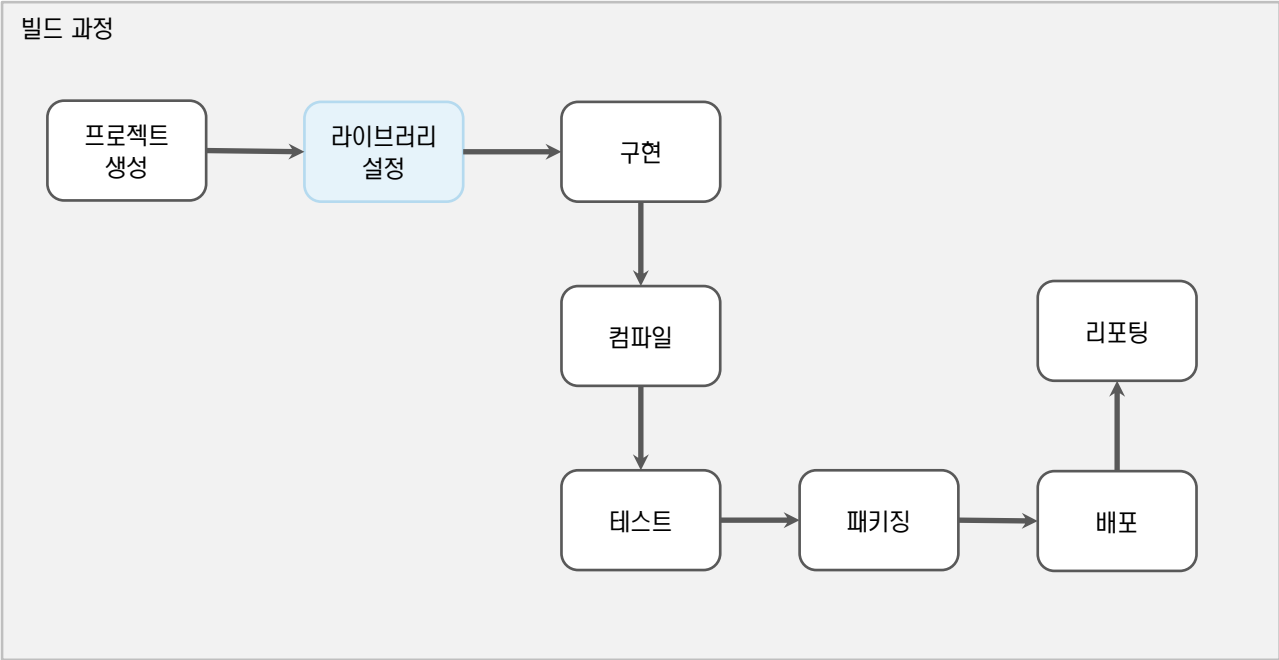
- ✓ Spring IoC 컨테이너는 인스턴스의 생성과 삭제 시점에 호출되는 메소드의 설정이 가능합니다.
- ✓ 빈의 초기화 메소드는 DI를 통해 모든 프로퍼티가 세팅된 후에만 가능한 초기화를 지원해줍니다.
- ✓ 빈 소멸 메소드는 컨테이너가 종료될 때 호출되어 종료 전에 처리해야 하는 작업을 수행합니다.



2.5 Maven 개요(1/3)

- ✓ 메이븐은 프로젝트 관리를 위한 도구로 프로젝트의 생성, 배포 및 리포팅까지 전체 과정을 포함합니다.
- ✓ 메이븐을 이용하면 해당 프로젝트에서 사용하는 라이브러리에 대한 종속성 관리를 편리하게 할 수 있습니다.
- ✓ 메이븐을 이용한 프로젝트 관리는 프로젝트의 버전 관리는 물론 프로젝트 구조의 일관성을 부여합니다.
- ✓ 메이븐은 컴파일, 테스트, 배포까지 자동화된 빌드 프로세스를 제공합니다.

Maven is ...
Maven is a build tool.
Dependency management tool.
Project management tool.
Standardized approach to building software.
Command line tool.
IDE integration.



2.5 Maven 개요(2/3) – Lifecycle

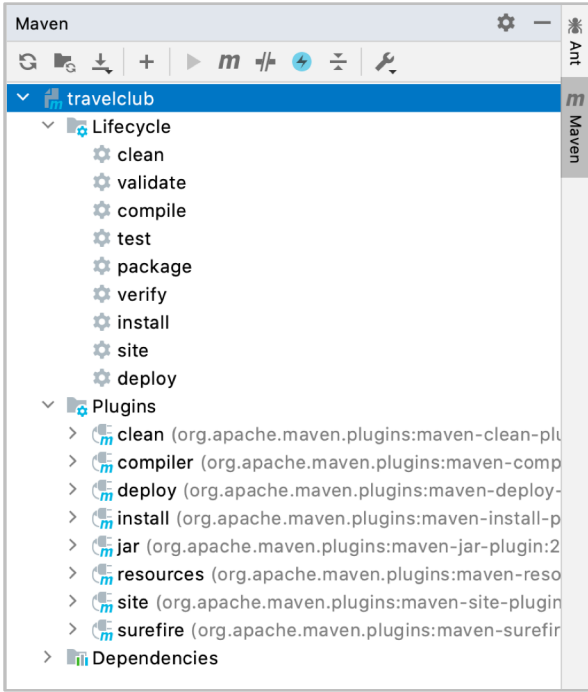
- ✓ 빌드 과정은 여러 단계로 구분되며 메이븐은 이 단계를 미리 정의하고 있으며 이를 라이프사이클이라고 합니다.
- ✓ 라이프사이클의 간 단계는 페이즈(Phase)라고 하며 각 페이즈는 플러그인(Plugin)을 기반으로 동작합니다.
- ✓ 하나의 플러그인은 여러 작업을 수행할 수 있으며 실행 가능한 각각의 작업을 골(Goal)이라고 합니다.
- ✓ 기본 라이프사이클은 페이즈 기반으로 빌드하며, 페이즈 실행시 연결된 플러그인의 골을 실행하는 구조로 동작합니다.

```
>mvn help:describe -Dcmd=compile

* validate: Not defined
* initialize: Not defined
* generate-sources: Not defined
* process-sources: Not defined
* generate-resources: Not defined
* process-resources: org.apache.maven.plugins:maven-resources-plugin:2.6:resources
* compile: org.apache.maven.plugins:maven-compiler-plugin:3.1:compile
* process-classes: Not defined
* generate-test-sources: Not defined
* process-test-sources: Not defined
* generate-test-resources: Not defined
* process-test-resources: org.apache.maven.plugins:maven-resources-plugin:2.6:testResources
* test-compile: org.apache.maven.plugins:maven-compiler-plugin:3.1:testCompile
* process-test-classes: Not defined
* test: org.apache.maven.plugins:maven-surefire-plugin:2.12.4:test
* prepare-package: Not defined
* package: org.apache.maven.plugins:maven-jar-plugin:2.4:jar
* pre-integration-test: Not defined
* integration-test: Not defined
* post-integration-test: Not defined
* verify: Not defined
* install: org.apache.maven.plugins:maven-install-plugin:2.4:install
* deploy: org.apache.maven.plugins:maven-deploy-plugin:2.7:deploy
```

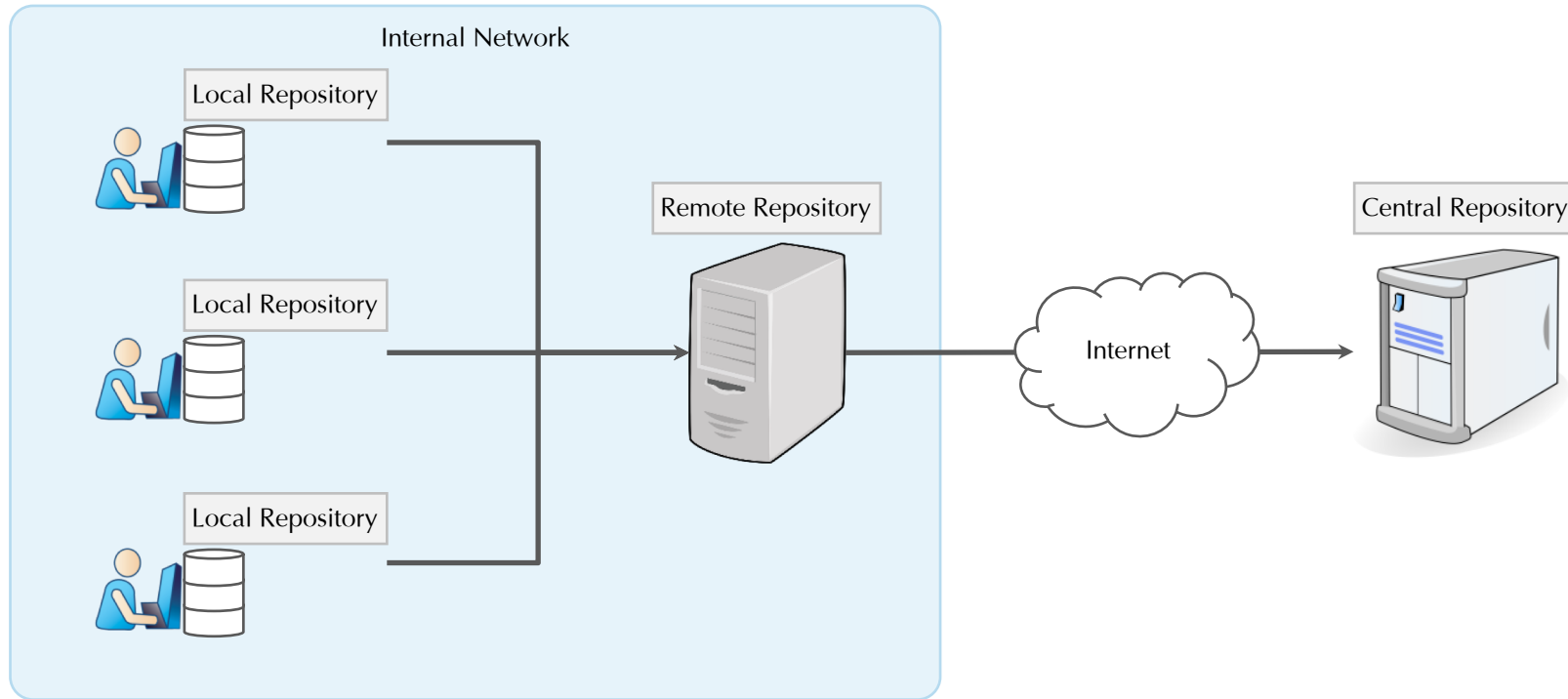
Phase : Plugin : Goal

process-resources: org.apache.maven.plugins:maven-resources-plugin:2.6:resources



2.5 Maven 개요(3/3) – Dependency Management

- ✓ 의존 라이브러리 관리 기능은 라이프사이클과 마찬가지로 메이븐의 중심이 되는 기능 중 하나입니다.
- ✓ 라이브러리 관리를 위한 저장소는 중앙(Central)저장소, 원격(Remote) 저장소, 로컬(Local) 저장소로 구분됩니다.
- ✓ 중앙 저장소는 오픈 소스 라이브러리, 플러그인, 아키타입을 관리하는 저장소입니다.
- ✓ 원격 저장소는 내부망에 구축하는 저장소, 혹은 중앙 저장소에 없는 별개의 라이브러리 저장소를 뜻합니다.
- ✓ 로컬 저장소는 개발자 컴퓨터의 저장소로 `USER_HOME/.m2/repository`의 위치를 갖습니다.





3.Spring Boot

- 3.1 Spring Boot 개요
- 3.2 Spring Boot Starter
- 3.3 Auto Configuration
- 3.4 Actuator
- 3.5 REST API 개요
- 3.6 REST API Naming

3.1 Spring Boot 개요(1/3)

- ✓ 스프링 부트는 단독 실행할 수 있고, 상용화 수준의, 실행 가능한 스프링 기반 애플리케이션을 쉽게 생성할 수 있게 합니다.
- ✓ 대부분의 스프링 부트 애플리케이션들은 적은 설정을 필요로 합니다.
- ✓ `java -jar` 커맨드 또는 전통적인 `war`로 배포 가능한 자바 애플리케이션 생성에 스프링 부트를 사용할 수 있습니다.
- ✓ 내장된 Tomcat, Jetty, Undertow를 이용해 WAR 배포없이 웹 애플리케이션을 실행할 수 있습니다.
- ✓ 많이 사용하는 라이브러리를 모아놓은 Starter POM 파일을 제공하여 쉽게 라이브러리를 관리할 수 있습니다.

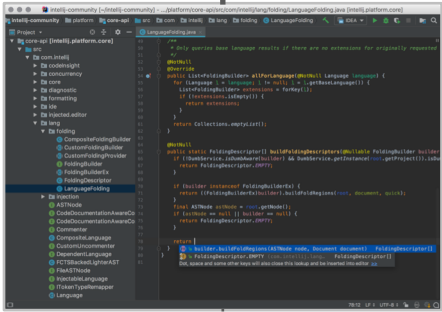


3.1 Spring Boot 개요(2/3)

Spring Web

MyShop.war

Apache Tomcat

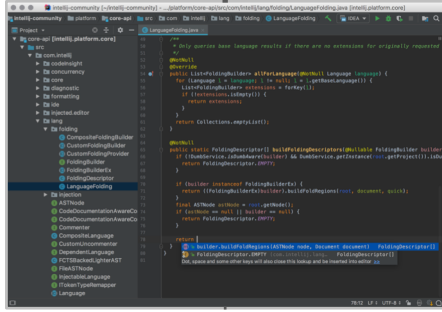


```
<dependencies>
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-web</artifactId>
  <version>5.3.4</version>
</dependency>
...
</dependencies>
```

Spring Boot

MyShop.jar

Apache Tomcat



```
<dependencies>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
</dependencies>
```


3.1 Spring Boot 개요(3/3)

- ✓ 스프링부트는 스프링 프레임워크에서 제공하는 모듈/기능의 복잡한 설정을 사용하지 않는 새로운 전략을 제공합니다.
- ✓ 스프링부트는 모든 것이 쉽게 오버라이드 될 수 있으면서 어떤 것도 오버라이드 할 필요가 없습니다.
- ✓ @SpringBootApplication는 @Configuration + @EnableAutoConfiguration + @ComponentScan 입니다.

Spring Boot Starters	Automatic configuration	Spring Boot Actuator
✓ Web MVC, JDBC, ORM 처럼 스프링 프로젝트의 모듈입니다. ✓ 스프링 애플리케이션에서 starter의 추가만으로 필요한 라이브러리가 포함되도록 합니다. ✓ 예를들어 RESTful Service 발행을 목적으로 Spring Web MVC를 사용하기 위해서 spring-boot-starter-web 종속성을 추가하기만 하면 됩니다. ✓ starter는 종속성의 개수를 줄여줄 뿐만 아니라 빌드하고자 하는 특별한 기능을 추가해 줍니다.	✓ 애플리케이션 기능에 대한 자동 설정을 제공합니다. ✓ Auto-configuration는 classpath를 이용합니다. ✓ 예를들어, JPA Starter(spring-boot-starter-data-jpa)를 종속성에 추가하면 JPA와 관련된 설정을 자동으로 시도합니다. ✓ 먼저 classpath에서 설정 정보를 찾고 설정할 수 없으면 application의 설정 정보에서 찾습니다.	✓ 스프링 애플리케이션을 모니터링하기 위한 production 레벨의 feature를 제공합니다. ✓ 이 feature는 HTTP 엔드포인트나 JMX로 모니터링 할 수 있습니다.
✓ spring-boot-starter-web은 아래 종속성을 자동으로 추가합니다. • spring-web-*.jar • spring-webmvc-*.jar • tomcat-*.jar • jackson-databind-*.jar	✓ dependency를 추가하지 않으면 스프링부트는 자동으로 설정할 수 없습니다. ✓ dependency management tool이 필요한 종속성을 제시합니다. ✓ Spring boot parent와 starter는 더 쉽게 설정할 수 있도록 해 줍니다. ✓ Maven 뿐만 아니라 Gradle, Ant/Ivy도 지원합니다.	✓ 애플리케이션 컨텍스트의 상세한 설정 ✓ 자동으로 설정된 설정 정보 ✓ 모든 환경 변수, 시스템 속성, 설정 속성, command-line arguments ✓ 메모리 사용량, GC, Web Request, data source 사용량 등의 다양한 metric 정보 ✓ 최근 처리된 HTTP 요청의 추적 ✓ Thread의 상태 정보

3.2 Spring Boot Starters

- ✓ Spring Boot는 Spring Boot Starter를 사용하여 프로젝트의 의존성을 쉽게 관리합니다.
- ✓ Spring Boot Starter란 기능 유형 별로 의존성과 설정을 자동화 해준 모듈입니다.
- ✓ pom.xml 파일 dependency에 Spring Boot Starter를 추가하여 관련 라이브러리들을 사용할 수 있습니다.

주요 Spring Boot Starter	설명
spring-boot-starter-web	웹 & REST 애플리케이션
spring-boot-starter-test	Unit 테스트, 통합 테스트
spring-boot-starter-jdbc	전통적인 JDBC
spring-boot-starter-security	Spring Security를 사용한 인증, 인가
spring-boot-starter-data-jpa	Hibernate 사용하는 Spring Data JPA
spring-boot-starter-actuator	운영 단계의 애플리케이션 모니터링과 관리를 도와주는 Spring Boot Actuator 지원
spring-boot-starter-mail	Java Mail과 Spring 프레임워크의 이메일 전송 지원

3.3 Auto Configuration

- ✓ 스프링부트 auto-configuration은 각 모듈별로 오버라이드 가능한 미리 정의된 설정값을 사용합니다.
- ✓ 예를들어 in-memory 데이터베이스인 H2 종속성을 추가하면 추가적인 설정이 필요하지 않습니다.
- ✓ 스프링부트는 스프링 애플리케이션의 classpath, yml/properties 파일, 미리 정의된 값 순서로 설정을 적용합니다.
- ✓ 스프링부트는 1,000개 이상의 설정 정보를 제공하며 개발자는 이 속성을 변경할 수 있습니다.

1	Home directory에 global settings에 정의된 설정을 적용한다.
2	@TestPropertySource 어노테이션에 정의된 속성을 적용한다.
3	Command-line arguments 속성을 적용한다.
4	SPRING_APPLICATION_JSON에 정의된 속성을 적용한다.
5	ServletConfig init 파라미터를 적용한다.
6	ServletContext init 파라미터를 적용한다.
7	Java:comp/env로 선언된 JNDI 속성을 적용한다.
8	Java System 속성을 적용한다.
9	OS 환경 변수를 적용한다.
10	application.properties or yml 파일을 적용한다.

Auto-Configuration 오버라이딩 순서

3.4 Actuator

- ✓ 스프링 부트 어플리케이션의 모니터링과 관리는 돕는 모듈이 바로 Actuator 입니다.
- ✓ Http, 혹은 JMS를 이용해 다양한 endpoints를 설정하고 어플리케이션의 상태를 확인할 수 있습니다.
- ✓ 스프링부트 1.x의 actuator는 4개(health, info, metrics, trace)의 Endpoint를 제공했습니다.
- ✓ 2.x actuator는 1.x의 의도를 그대로 유지하면서 설계를 더 단순화하고 기능을 더 확장하였습니다.

/actuator/auditevents	로그인/로그아웃과 같은 보안 로그와 관련된 이벤트
/actuator/beans	BeanFactory에 있는 사용 가능한 모든 bean 목록
/actuator/conditions	Auto-configuration 정보
/actuator/configprops	@ConfigurationProperties로 설정되고 조회 가능한 모든 목록
/actuator/env	현재 환경 설정 프로퍼티 목록
/actuator/health	애플리케이션의 health status
/actuator/heapdump	애플리케이션 JVM의 heapdump
/actuator/info	애플리케이션의 일반적인 정보
/actuator/loggers	애플리케이션 logger의 log level 변경 및 조회
/actuator/metrics	애플리케이션의 상세한 metrics
/actuator/sessions	HTTP session 목록

3.5 REST API 개요

- ✓ API(Application Programming Interface)는 애플리케이션의 데이터와 기능을 외부에 공개하는 방법이라 할 수 있으며, 사용자는 API로 제공되는 기능의 구현 방법에 상관없이 해당 기능을 사용할 수 있습니다.
- ✓ REST API는 웹상에서 사용되는 여러 리소스(resource)를 HTTP URI로 표현하고, 해당 리소스에 대한 행위를 HTTP Method로 정의하는 방식입니다.
- ✓ REST API 설계는 아키텍처 스타일을 의미하며 표준은 아니며 적용에 유연한 입장을 취할 수 있습니다.
- ✓ API 설계의 핵심은 실용주의 관점을 유지하는 REST 입니다.



3.6 REST API Naming (1/2)

- ✓ 실용적인 RESTful 디자인의 첫번째 원칙은 간결하고 직관적인 기준 URL을 유지하는 것입니다.
- ✓ 기준 URL 설계는 API에서 가장 중요한 설계 행동유도성(affordance) 입니다.
- ✓ 행동유도성은 가이드 문서가 필요 없는 설계 재산입니다. ← 직관적인 설계



설계 행동유도성(affordance) 과 문서 간의 충돌 !!

3.6 REST API Naming (2/2) – 명사와 동사

- ✓ 자원(resource) 별로 두 개의 기준 url을 사용합니다.
- ✓ 기준 url에는 동사를 두지 않습니다.
- ✓ 컬렉션이나 요소들을 다룰 때는 HTTP 메소드를 사용합니다.

```
// 목록을 위한 URL
/articles

// 목록 중 특정 개체를 위한 URL
/articles/1234
```

Resource	POST(create)	GET(read)	PUT(update)	DELETE(delete)
/articles	새로운 article 생성	article 목록	articles에 대한 대량 업데이트	모든 articles 삭제
/articles/1234	에러	특정 article 보기	있으면 업데이트, 없으면 에러	삭제



4.Spring MVC

4.1 웹 프레임워크 아키텍처

4.2 Spring MVC 개요

4.3 DispatcherServlet

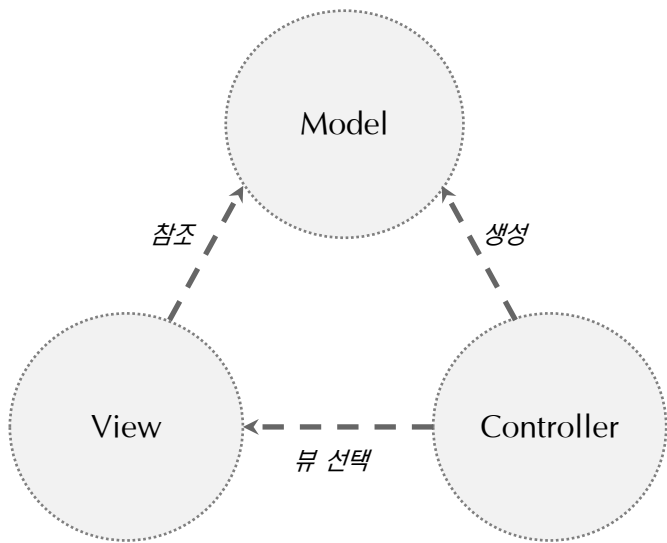
4.4 Controller

4.5 View

4.6 Model

4.1 웹 프레임워크 아키텍처(1/2) – MVC 패턴

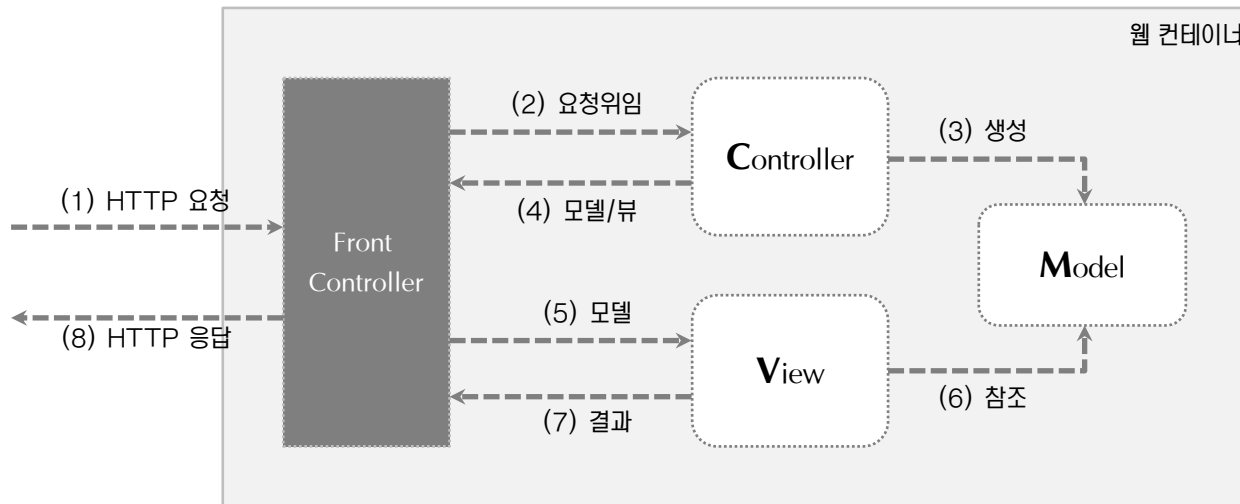
- ✓ MVC(Model-View-Controller) 패턴은 코드를 처리하는 영역을 Model, View, Controller로 분리합니다.
- ✓ 영역별로 분리하면 컴포넌트 간의 결합도가 낮아져, 컴포넌트 변경이 다른 영역 컴포넌트에 영향을 주지 않습니다.
- ✓ 사용자 화면을 비즈니스 로직과 분리하므로, 기능 확장과 변경이 편리합니다.
- ✓ 개발 과정이 복잡해 초보자가 이해하고 개발하기에 다소 어려워, 처음엔 개발 진도가 늦어질 수 있습니다.



Model	- 어플리케이션 상태의 캡슐화 - 상태 쿼리에 대한 응답 - 어플리케이션의 기능 표현 - 변경을 뷰에 통지
View	- 모델을 화면에 시각적으로 표현 - 모델에게 업데이트 요청 - 사용자의 입력을 컨트롤러에 전달 - 컨트롤러가 뷰를 선택하도록 허용
Controller	- 어플리케이션의 행위 정의 - 사용자 액션에 대한 모델 업데이트와 매핑 - 응답에 대한 뷰 선택

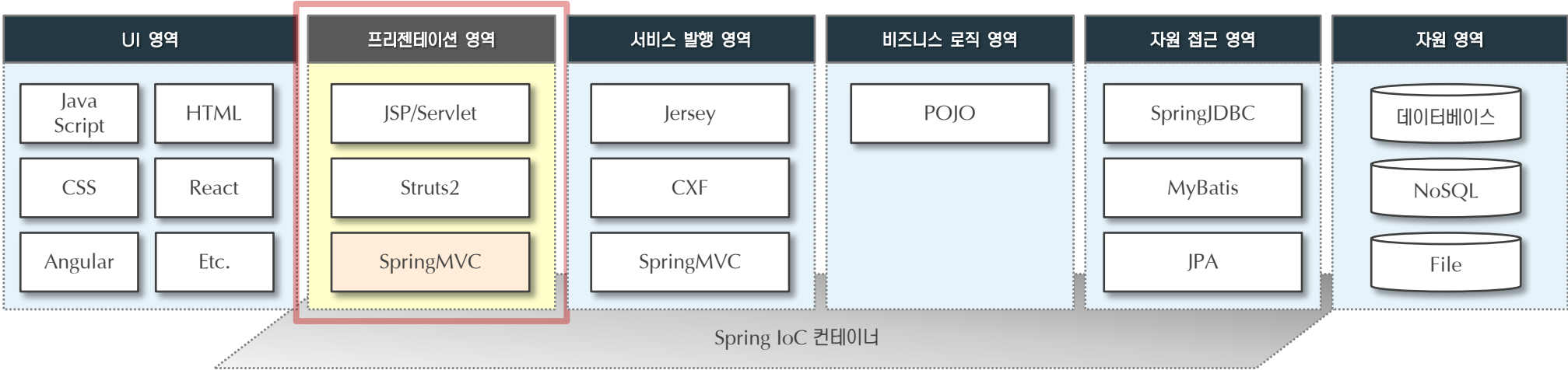
4.1 웹 프레임워크 아키텍처(2/2) – Front Controller 패턴

- ✓ 웹 MVC 프레임워크는 J2EE 패턴 의 Front Controller 패턴에 기초합니다.
- ✓ Front Controller는 클라이언트의 모든 요청을 받아 해당 요청의 처리를 담당하는 컴포넌트에 처리를 위임합니다.
- ✓ 프리젠테이션 계층의 제일 앞에서 모든 요청을 최초로 수신하여 처리하는 서블릿을 Front Controller라고 합니다.
- ✓ 웹 MVC 프레임워크는 요청을 각 컨트롤러로 분기하는 중앙 서블릿 중심으로 설계되었습니다.



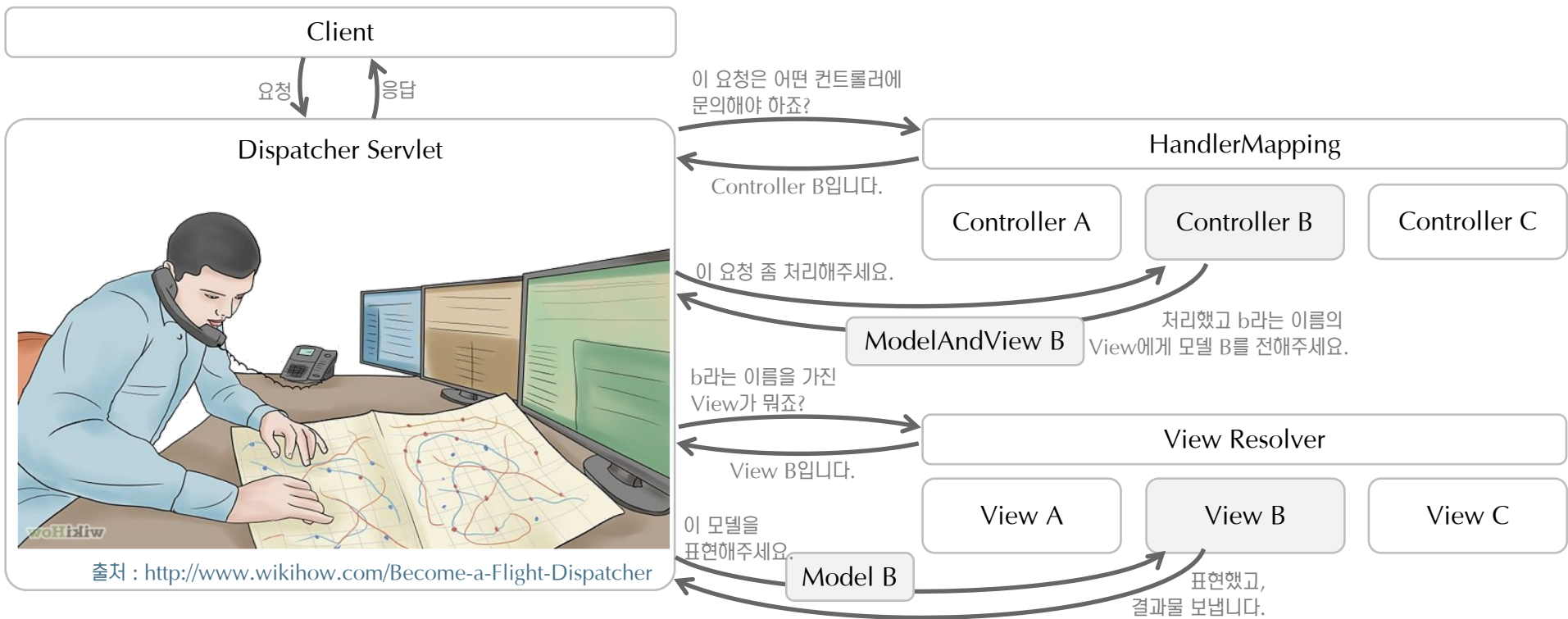
4.2 Spring MVC 개요

- ✓ Spring MVC는 스프링에서 프리젠테이션 계층을 담당하는 서블릿 기반 MVC 프레임워크입니다.
- ✓ Spring MVC는 다른 웹 프레임워크에 비해 특정 클래스 상속, 참조 또는 구현 제약사항이 적습니다.
- ✓ Spring은 POJO(Plain Old Java Object)를 지향하므로 복잡한 설정 없이 비즈니스 로직에 집중할 수 있습니다.
- ✓ Spring IoC 컨테이너를 사용하여 웹 프레임워크 연계를 위한 추가 설정 없이 Spring MVC를 사용할 수 있습니다.



4.3 DispatcherServlet (1/2) – 요청 처리 절차

- ✓ DispatcherServlet은 Spring MVC 의 핵심으로써, 프론트 컨트롤러 역할을 담당합니다.
- ✓ 클라이언트의 요청을 받아 응답하는 과정에서 담당할 대상을 선택하고 역할을 분배하는 등의 작업을 수행합니다.
- ✓ DispatcherServlet이 요청을 처리하는 과정을 이해하는 것이 Spring MVC 이해의 핵심입니다.
- ✓ 웹 애플리케이션 설정파일(web.xml)에 서블릿 요청을 DispatcherServlet 클래스가 처리하도록 매핑 합니다.



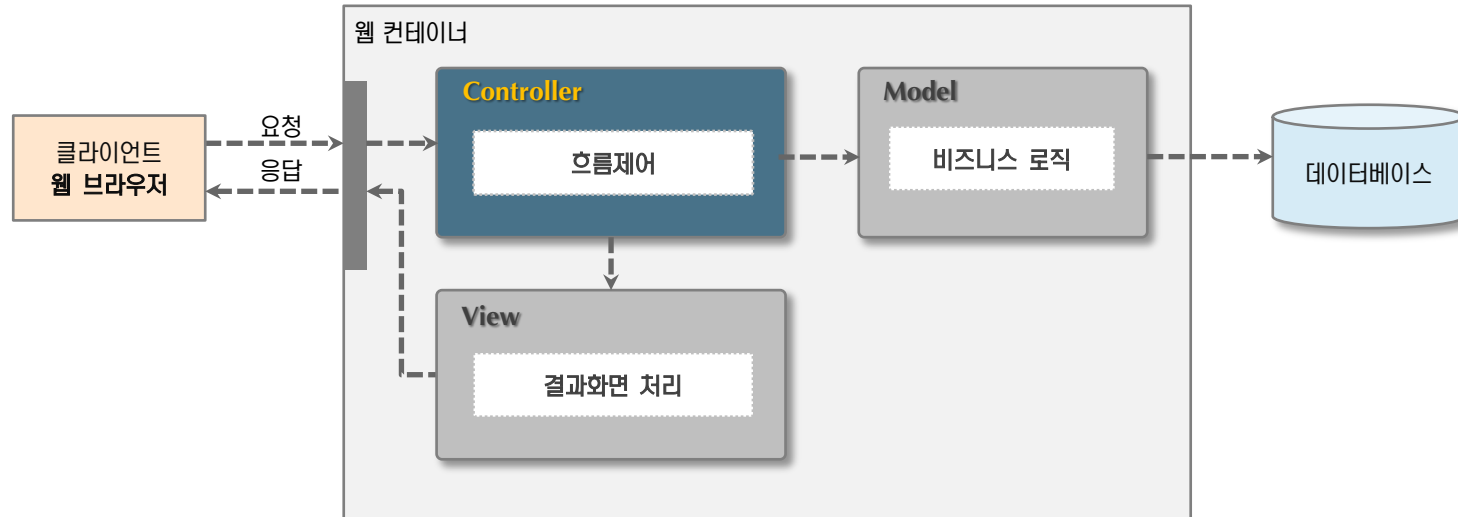
4.3 DispatcherServlet (2/2) – 요청 처리 절차

- ✓ 개발자는 Controller만 구현하고, 그 외의 객체는 Spring MVC가 제공하는 클래스를 사용합니다.
- ✓ DispatcherServlet은 요청에서부터 응답까지의 전체 라이프사이클을 관리합니다.
- ✓ HandlerMapping이 요청을 처리할 컨트롤러를 결정하는 기준에는 URL, 클래스명, 어노테이션 등이 있습니다.
- ✓ ModelAndView는 요청처리 결과 데이터와 화면에 표시할 View이름을 포함합니다.

클래스 명	설명
DispatcherServlet	단일 프론트 컨트롤러로 모든 HTTP 요청을 수신하여 그 밖의 오브젝트 사이의 흐름을 제어합니다.
HandlerMapping	클라이언트가 요청한 URL을 바탕으로 어느 컨트롤러를 실행할 지 결정합니다. URL, 컨트롤러 클래스명, 어노테이션등을 기준으로 결정합니다.
Controller	클라이언트 요청에 맞는 프리젠테이션 층의 애플리케이션 처리를 실행합니다. 요청처리 결과 데이터를 ModelAndView에 반영합니다.
ModelAndView	Model은 컨트롤러에서 뷰에 전달할 데이터를 저장하는 객체입니다. ModelAndView는 실제 View의 JSP정보를 갖고 있지 않으며, ViewResolver가 논리적 이름을 실제 JSP이름으로 변환합니다.
ViewResolver	View 이름을 바탕으로 View 객체를 결정합니다.
View	View는 화면에 표시하도록 요청합니다.

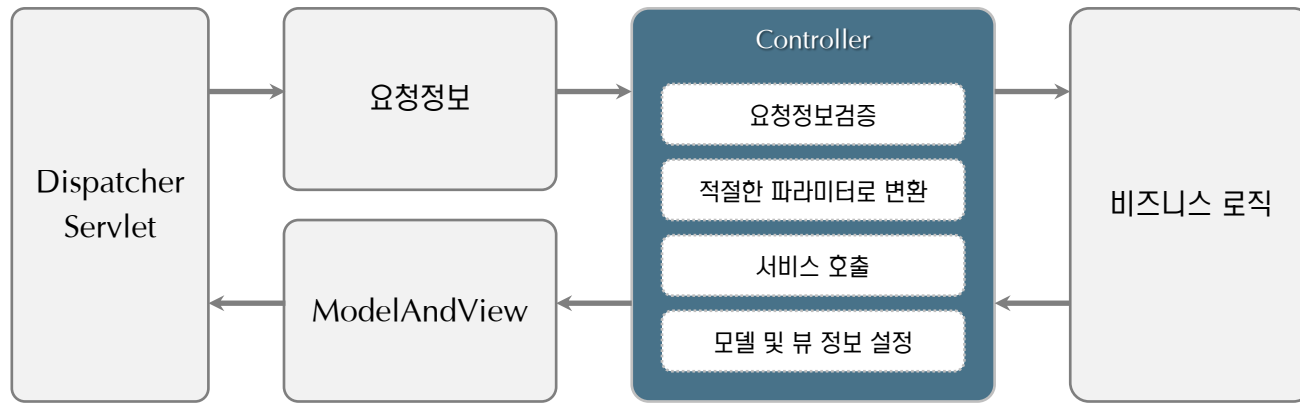
4.4 Controller(1/6) – 개요(1/2)

- ✓ 컨트롤러는 MVC (Model, View, Controller) 중 가장 많은 작업을 처리합니다.
- ✓ 컨트롤러는 클라이언트의 모든 요청을 받아서 처리합니다.
- ✓ 컨트롤러는 요청 내용을 처리하는 서비스를 호출하여 비즈니스 로직을 처리합니다.
- ✓ 컨트롤러는 처리결과를 View에게 전달하여 결과 화면을 생성하게 합니다.



4.4 Controller(2/6) – 개요(2/2)

- ✓ Spring MVC 컨트롤러는 DispatcherServlet 에게 전달받은 요청 정보의 정합성을 검증합니다.
- ✓ 서비스에게 비즈니스 로직 처리를 위임합니다. 이를 위해 적절한 파라미터로 변환하여 서비스에게 전달합니다.
- ✓ 서비스로 부터 처리결과를 받으면 어떤 뷰를 보여줘야 할지 결정하고, 처리 결과를 뷰에게 전달할 형태로 생성합니다.
- ✓ 모델과 뷰를 생성하여 DispatcherServlet에게 전달합니다.



4.4 Controller(3/6) – @Controller

- ✓ @Controller 어노테이션을 클래스에 등록하여 XML 빈 등록 설정 없이 컨트롤러를 빈으로 등록합니다.
- ✓ 단, 어노테이션을 사용하기 위해서는 설정파일에 <component-scan> 요소를 추가합니다.
- ✓ <component-scan> 요소를 추가하고 어노테이션을 스캔할 범위를 패키지 단위로 지정합니다.
- ✓ 스캔할 범위를 세밀하게 설정하려면, 하위 엘리먼트로 <include-filter> 를 추가합니다.

```
@Controller
public class HomeController {

    @RequestMapping(value = "/", method = RequestMethod.GET)
    public String home(Locale locale, Model model) {
        Date date = new Date();
        DateFormat dateFormat = DateFormat.getDateInstance(DateFormat.LONG, DateFormat.LONG, locale);

        String formattedDate = dateFormat.format(date);

        model.addAttribute("serverTime", formattedDate );

        return "home";
    }
}
```

컨트롤러 클래스

```
<context:component-scan base-package="com.namoo.web">
    <context:include-filter type="annotation" expression="org.springframework.stereotype.Controller" />
</context:component-scan>
```

스프링 설정파일

4.4 Controller(4/6) – @Controller

- ✓ 핸들러 매핑은 HTTP 요청정보를 처리하는 컨트롤러(핸들러)를 찾아주는 역할을 합니다.
- ✓ 핸들러 매핑을 여러 개 등록하여 사용하는 경우, `order`를 이용해 우선 순위를 정할 수 있습니다.
- ✓ 핸들러 매핑 빈의 `defaultHandler` 프로퍼티에 URL을 매핑 하지 못할 때 사용할 디폴트 핸들러를 지정할 수 있습니다.
- ✓ 스프링에서는 다섯 가지의 핸들러 매핑 전략 클래스를 제공합니다.

```
<bean id="homeController" class="com.namoo.web.HomeController" />

<bean class="org.springframework.web.servlet.handler.BeanNameUrlHandlerMapping">
    <property name="defaultHandler" ref="homeController"/>
</bean>
```

스프링 설정파일

4.4 Controller(5/6) – Request Mapping(1/2)

- ✓ RequestMappingHandlerMapping은 @RequestMapping 정보로 컨트롤러를 매핑합니다.
- ✓ 부모 컨트롤러 클래스에 @RequestMapping를 추가하면 하위 컨트롤러 클래스에도 적용됩니다.
- ✓ 단, 하위 클래스에서 @RequestMapping을 재정의하면 상위 클래스의 @RequestMapping은 무시됩니다.
- ✓ @RequestMapping을 인터페이스에 작성한 경우도 구현 클래스에 동일하게 적용됩니다.

```
public class ParentController {  
    @RequestMapping("find")  
    public String find() {  
        return "find";  
    }  
}
```

```
@Controller  
public class ChildController extends ParentController {  
    @Override  
    public String find() {  
        System.out.println("child find");  
        return "find";  
    }  
}
```



ChildController 는 상위 클래스인 **ParentController**에 정의된 **@RequestMapping**를 그대로 상속받습니다.
결국 /find URL을 입력했을 때, "child find " 문구가 출력됩니다.

4.4 Controller(6/6) – Request Mapping (2/2)

- ✓ @RequestMapping을 이용하면 URL 뿐 아니라 여러 요청정보를 활용할 수 있습니다.
- ✓ value 속성은 매핑할 URL을 설정합니다.
- ✓ method 속성은 요청을 매핑할 HTTP Method를 설정합니다. (GET, POST, PUT, DELETE 등)
- ✓ params 속성은 요청 파라미터를 설정합니다. 파라미터 값이 지정한 값과 일치할 때만 요청을 매핑합니다.

```
@RequestMapping(value="/boards/{boardId}", method=RequestMethod.GET, params="admin=true")
public String findBoard(@PathVariable("boardId") String boardId, Model model){

    .....

    return "/board/read";
}
```



/boards/{boardId} 로 요청된 GET 방식의 요청 중 admin 파라미터 값이 true인 요청만 매핑합니다.

4.5 View (1/5) – 개요

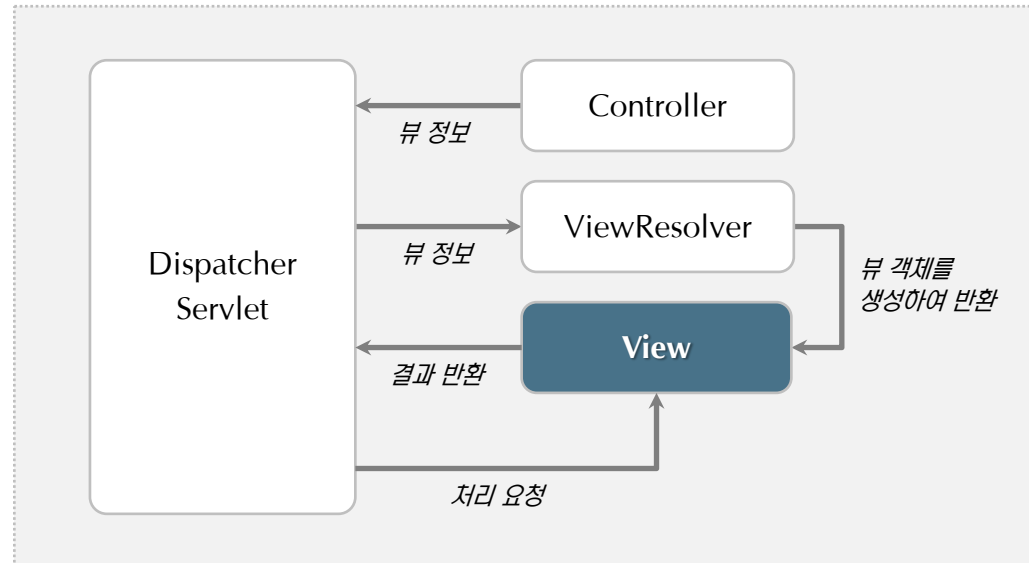
- ✓ MVC에서 뷰(View)는 모델을 전달 받아 모델의 정보를 다양한 형식으로 표현하는 기능을 담당합니다.
- ✓ 일반적으로 뷰는 HTML로 생성되어 브라우저가 결과를 표현하지만, 엑셀/PDF/XML 등의 콘텐츠로 생성되기도 합니다.
- ✓ 스프링에서 제공하는 뷰를 사용하여 여러 콘텐츠를 작성할 수도 있습니다.
- ✓ 뷰를 직접 사용하는 대신, 메시지 컨버터를 사용하면 XML, JSON 등을 생성할 수 있습니다.

```
public class Posting {  
  
    private String id;  
    private String title;  
    private String contents;  
  
    public Posting () {}  
  
    public Posting(String id, String title, String contents) {  
        this.id = id;  
        this.title = title;  
        this.contents = contents;  
    }  
}
```



4.5 View (2/5) – View & Controller

- ✓ MVC에서는 뷰와 컨트롤러의 연결 방식에 대한 이해가 필요합니다.
- ✓ 컨트롤러가 처리 후 돌아갈 뷰를 지정해주었다면 `DispatcherServlet`은 해당 뷰에 처리를 요청합니다.
- ✓ 명확히 뷰 객체를 지정하지 않고, 뷰 이름만 알려주어도 `ViewResolver`를 통해 뷰 객체를 생성할 수 있습니다.
- ✓ 어떠한 뷰 정보도 알려주지 않을 때, `DefaultRequestToViewNameTranslator`를 통해 뷰 이름을 설정할 수 있습니다.



4.5 View (3/5) – View 지정 방법

- ✓ 뷰를 지정하는 방법에는 컨트롤러가 명시적으로 리턴하는 방법과, 설정에 의해 자동으로 지정하는 방법 2가지가 있습니다.
- ✓ 첫 번째는 컨트롤러가 ModelAndView View 객체에 뷰 이름을 설정하거나, 문자열로 뷰 이름을 반환하는 방법입니다.
- ✓ 두 번째는 컨트롤러가 Model, Map, void를 리턴하는 경우, 설정에 의해 자동으로 뷰를 지정하는 방법입니다.

[명시적인 방법] 뷰 이름을 담은 ModelAndView 객체를 리턴함

```
@Controller
public class BoardController {

    @RequestMapping("/board")
    public ModelAndView list() {
        return new ModelAndView("board");
    }
}
```

[명시적인 방법] 뷰 이름을 문자열로 리턴함

```
@Controller
public class BoardController {

    @RequestMapping("/board")
    public String list() {
        return "board";
    }
}
```

[자동지정 방법] 요청 URI 로 부터 뷰 이름이 지정됨

```
@Controller
public class BoardController {

    @RequestMapping("/board.do")
    public Map<String, Object> list() {
        Map<String, Object> model = new HashMap<String, Object>();
        ...
        return model;
    }
}
```



자동으로 뷰 이름이 지정되는 경우, 요청한 URI 에서 맨 앞의 / 기호와 끝에 있는 확장자를 제외한 이름이 뷰의 이름이 됩니다. 따라서 이 예제에서는 board가 뷰 이름이 됩니다.

4.5 View (4/5) – ViewResolver

- ✓ 컨트롤러가 뷰 이름을 반환하면 ViewResolver를 통해 실제 사용할 뷰를 결정합니다.
- ✓ 뷰 이름은 실제 존재하는 뷰가 아니므로, ViewResolver를 통해 논리적인 뷰(뷰 이름)를 실질적인 뷰로 바꿔줍니다.
- ✓ ViewResolver는 기본 제공 ViewResolver 구현체를 등록하여 사용하거나 직접 확장하여 구현할 수 있습니다.
- ✓ 특정 ViewResolver를 빈으로 등록하지 않으면, InternalResourceViewResolver 가 자동으로 등록됩니다.

```
<beans:bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <beans:property name="prefix" value="/WEB-INF/views/" />
  <beans:property name="suffix" value=".jsp" />
</beans:bean>
```

스프링 설정파일



InternalResourceViewResolver 에 prefix/suffix 등의 속성을 지정하고 싶다면 빈으로 등록해야 합니다.
STS 플러그인을 통해 프로젝트를 생성한 경우, 이러한 설정이 기본으로 제공되므로 편리합니다.

4.5 View (5/5) – Redirect View

- ✓ 리다이렉트뷰는 실제 뷰를 생성하지 않고, URL만 생성하여 다른 페이지로 리다이렉트 합니다.
- ✓ 주로 컨트롤러에서 기능을 수행 후 기존에 존재하는 특정 요청으로 연결하고 싶을 때 사용합니다.
- ✓ 예를 들어 저장 후 목록조회로 연결할 때 리다이렉트를 사용할 수 있습니다.
- ✓ 리다이렉트 뷰는 **RedirectView** 객체나, 접두어 **redirect:** 를 포함한 문자열을 리턴하여 지정합니다.

```
// 레시피를 등록하고 전체 목록 조회화면으로 리다이렉트
@RequestMapping(value = "/recipe", method = RequestMethod.POST)
public String addRecipe(Recipe recipe) {

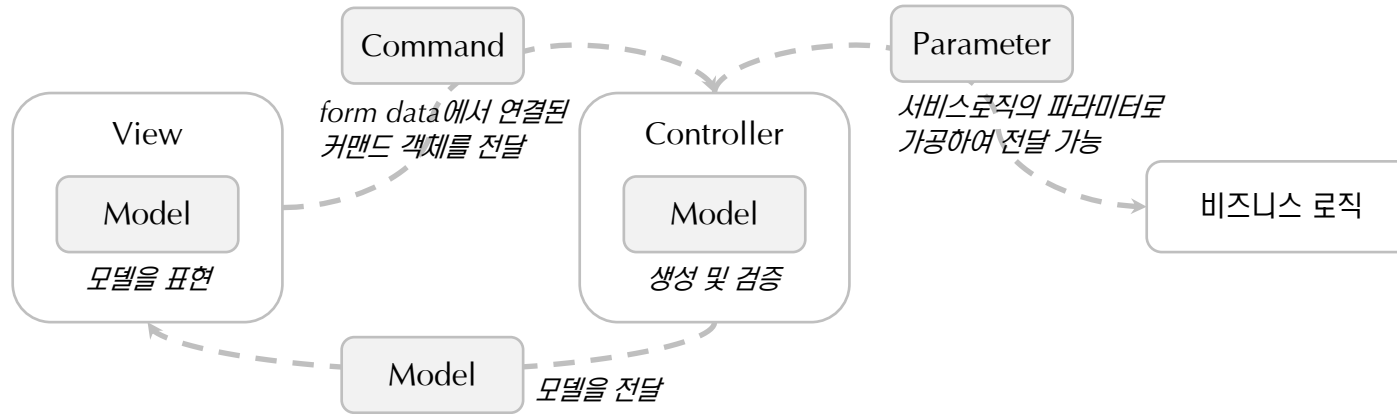
    chef.createRecipe(recipe);
    return "redirect:/";
}
```



RedirectView 오브젝트를 직접 사용하지 않아도, 문자열에 리다이렉트의 접두어를 포함시키면 ViewResolver에서 처리합니다. `UrlBasedViewResolver`에는 `redirect:`에 대한 처리가 구현되어 있습니다. 기본으로 제공하는 `InternalResourceViewResolver`는 `UrlBasedViewResolver` 클래스를 상속받기 때문에 특별한 설정을 하지 않아도 사용할 수 있습니다.

4.6 Model (1/4) – Spring MVC 모델

- ✓ 컨트롤러는 뷰에서 전달된 데이터를 커맨드 객체로 전달받아 처리할 수 있습니다.
- ✓ 컨트롤러에서 받거나 생성된 모델은 검증과정을 거친 후 비즈니스 로직 메소드에 전달하는 파라미터가 될 수 있습니다.
- ✓ 컨트롤러가 모델을 리턴하면 DispatcherServlet는 모델을 뷰에게 전달합니다.
- ✓ 뷰는 전달받은 모델을 클라이언트에게 적절한 형태로 표현합니다.



4.6 Model (2/4) – 커맨드 객체 (1/2)

- ✓ Spring MVC는 객체 속성과 HTML 폼으로 보낸 데이터의 이름이 같으면 자동으로 매핑해서 건네줍니다.
- ✓ 컨트롤러는 뷰의 Form으로 부터 전달된 데이터를 객체 파라미터로 받아 처리할 수 있습니다.
- ✓ 커맨드 객체는 모델에 추가하지 않아도 자동으로 모델에 추가되어 뷰에 전달됩니다.
- ✓ List 형태를 컨트롤러가 받기 위해서는 반드시 감싸주는 커맨드 객체가 있어야 합니다.

```
<form action="posting" method="post">
  <input type="text" name="postings[0].id" />
  <input type="text" name="postings[0].title" />
  <input type="text" name="postings[0].contents" />
  <br/>
  <input type="text" name="postings[1].id" />
  <input type="text" name="postings[1].title" />
  <input type="text" name="postings[1].contents" />
  <br/>
  <input type="submit" />
</form>
```

HTML 폼

```
public class PostingCommand {

    private List<Posting> postings;

    public void setPostings(List<Posting> postings) {
        this.postings = postings;
    }

}
```

커맨드 객체

```
@Controller
@RequestMapping("/posting")
public class PostingController {

    @RequestMapping(method=RequestMethod.POST)
    public String regist(PostingCommand command) {
        ...
    }
}
```

컨트롤러 클래스

4.6 Model (3/4) – 커맨드 객체 (2/2)

- ✓ EL 표기법을 이용하여 컨트롤러에서 받아온 커맨드 객체를 View에서 사용할 수 있습니다.
- ✓ `@ModelAttribute`를 이용하여 이름을 따로 정해주지 않으면 커맨드 객체의 클래스명을 사용합니다.
- ✓ 이름을 변경하고 싶다면, 해당 커맨드 객체 앞에 `@ModelAttribute`를 붙여서 이름을 지정합니다.

```
@Controller
public class BoardController {

    @RequestMapping(value="/board/regist", method=RequestMethod.POST)
    public String regist(Board command) {
        ...
    }
}
```

View에서는 커맨드 객체의 클래스명(첫 글자 소문자)을 사용하여 커맨드 객체에 접근합니다.

```
<body>
...
${board.title}
```

View 코드 (JSP)

```
@Controller
@RequestMapping("/board/regist")
public class BoardController {

    @RequestMapping(method=RequestMethod.POST)
    public String regist(@ModelAttribute("faq") Board command) {
        ...
    }
}
```

`@ModelAttribute`를 파라미터 앞에 붙여서 설정하면, View에서 사용하는 이름을 변경할 수 있습니다.

```
<body>
...
${faq.title}
```

View 코드 (JSP)

4.6 Model (4/4) – @RequestParam, @ModelAttribute

- ✓ @RequestParam 은 메소드 파라미터를 요청 파라미터에서 1:1로 받을 경우 사용합니다.
- ✓ @ModelAttribute 는 요청 파라미터를 객체 형태로 받기 위해 사용합니다.
- ✓ 검색 조건과 같이 여러 파라미터를 객체 형태로 받거나 폼 Submit으로 전달되는 파라미터를 객체로 받는데 유용합니다.
- ✓ @ModelAttribute가 붙은 객체는 자동으로 Model에 추가되므로 뷰에서 바로 사용할 수 있습니다.

```
@RequestMapping(method=RequestMethod.POST)
public String regist(@ModelAttribute("posting") Posting posting) {
    ...
    return "posting";
}
```



5.Spring Data JPA

5.1 영속성의 이해

5.2 객체의 세상과 테이블의 세상

5.3 Spring Data + JPA의 이해

5.4 Spring Data JPA 기초

5.1 영속성의 이해 (1/3)

- ✓ 영속성은 사전적인 의미로 영원히 계속되는 성질이나 능력을 뜻합니다.
- ✓ 물리적인 저장소(하드 디스크 또는 테이프 등)에 저장하는 행위를 영속화라 표현합니다.
- ✓ 자바에서 SQL을 통해 RDBMS에 데이터를 저장하는 행위를 영속화라 표현하기도 합니다.
- ✓ 객체지향 애플리케이션에서는 객체를 특정 저장소에 저장하는 행위를 영속화라 부릅니다.

```
public class UserJDBCProviderImpl implements UserProvider {
    private final String DB_URL = "jdbc:hsqldb:mem:test";
    private final String USER_ID = "sa";
    private final String PASSWORD = "";

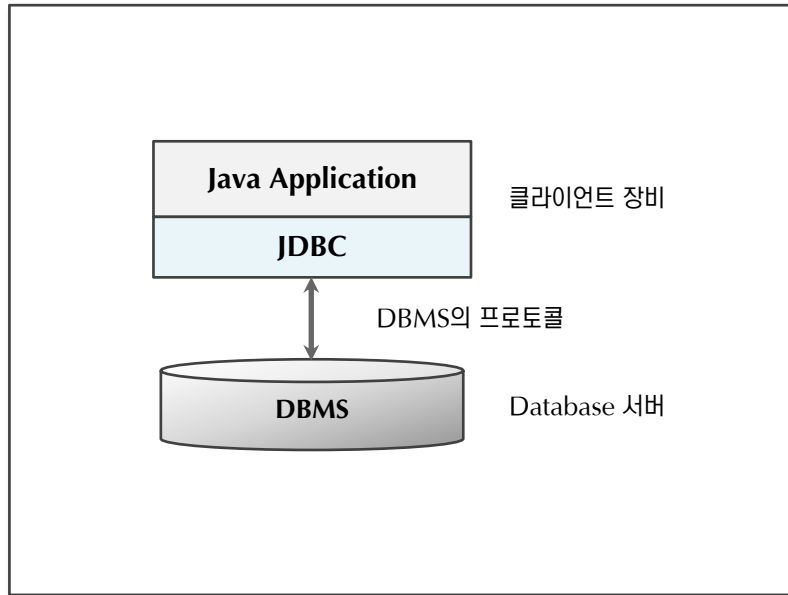
    public UserJDBCProviderImpl() {
        try { Class.forName("org.hsqldb.jdbcDriver"); }
        catch (ClassNotFoundException e) {
            throw new RuntimeException("JDBC 드라이버 로드 중 에러 발생"); } }

    public List<User> findAllUsers() {
        List<User> users = new ArrayList<User>();
        String sql = "SELECT USERID, NAME, EMAIL, TYPE "
            + "FROM USER_TB " + "ORDER BY USERID ASC";
        Connection conn = null;
        PreparedStatement psmt = null;
        ResultSet rs = null;
        try {
            conn = DriverManager.getConnection(DB_URL,
                USER_ID, PASSWORD);
            psmt = conn.prepareStatement(sql);
            rs = psmt.executeQuery();
```

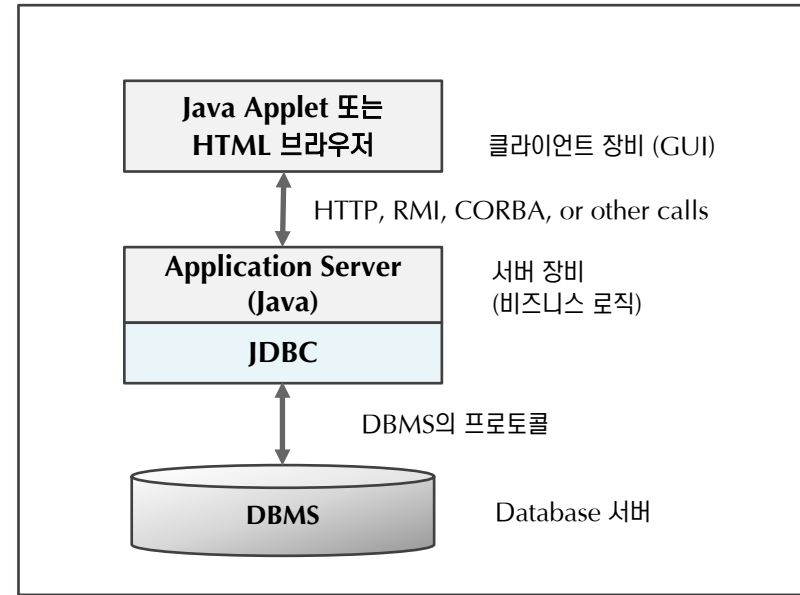
```
            while (rs.next()) {
                User user = new User();
                user.setUserId(rs.getString(1));
                user.setName(rs.getString(2));
                user.setName(rs.getString(3));
                user.setName(rs.getString(4));
                users.add(user);
            }
        } catch (SQLException e) {
            throw new RuntimeException("전체 사용자 조회중 오류 발생",
                e);
        } finally {
            try {
                if (rs != null) rs.close();
                if (psmt != null) psmt.close();
                if (conn != null) conn.close();
            } catch (SQLException e) {
            }
        }
        return users;
    }
}
```

5.1 영속성의 이해 (2/3)

- ✓ 자바에서는 다양한 방법으로 객체를 RDBMS로 영속화 합니다.
- ✓ 솔루션(TopLink 또는 Hibernate)을 사용해서 객체를 RDBMS로 영속화 할 수 있습니다.
- ✓ Enterprise JavaBeans의 엔티티빈을 사용해서 객체를 RDBMS로 영속화 할 수 있습니다.
- ✓ 자바에서는 객체를 RDBMS에 영속화 할 때 일반적으로 OR 맵핑 또는 SQL 맵핑을 사용합니다.



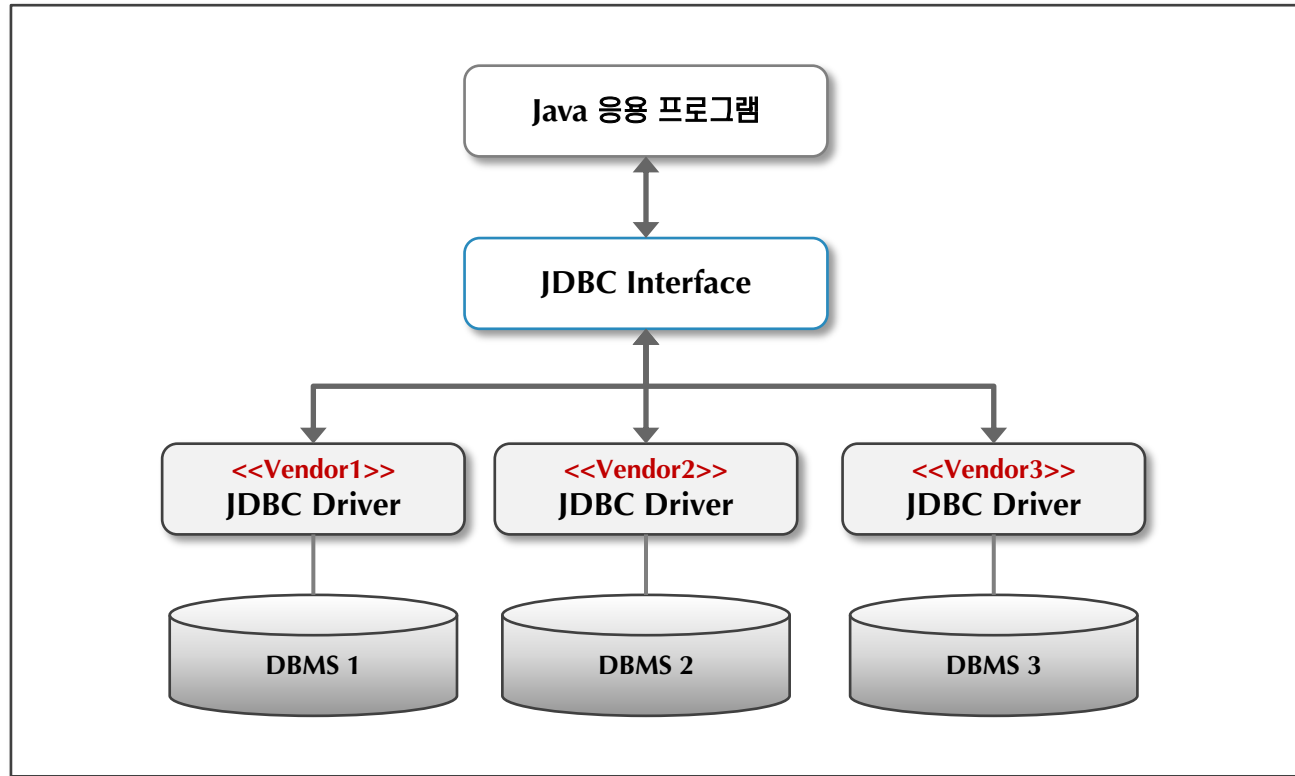
데이터 접근을 위한 2-Tire 아키텍처



데이터 접근을 위한 3-Tire 아키텍처

5.1 영속성의 이해 (3/3) – JDBC API

- ✓ 자바에서는 직접적으로 데이터베이스의 프로토콜을 사용하는 대신 JDBC 인터페이스를 사용 합니다.
- ✓ JDBC 인터페이스는 자바와 데이터베이스를 연결하기 위한 기능을 정의합니다.
- ✓ 데이터베이스 제조사는 JDBC 인터페이스를 자사 제품에 맞도록 구현합니다.
- ✓ 다양한 벤더들 때문에 자바에서는 JDBC 인터페이스 정의만 제공합니다.



JDBC 인터페이스와 DBMS의 관계

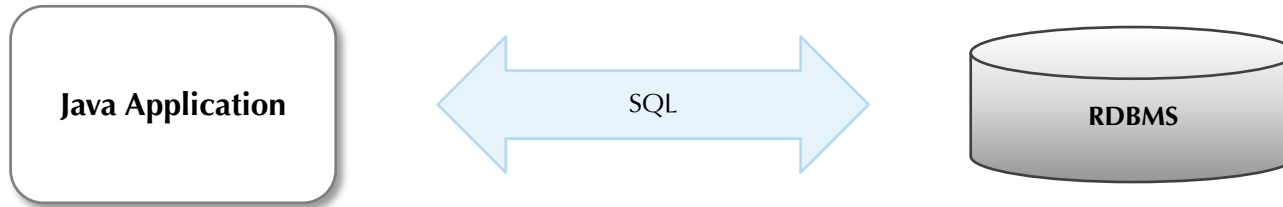
5.2 객체의 세상과 테이블의 세상 [1/2]

- ✓ 객체 중심의 객체지향 어플리케이션과 테이블 중심의 관계형 데이터베이스는 서로의 목표가 다릅니다.
- ✓ 이를 패러다임이 일치하지 않는다고 표현하며 이는 개발 과정에서 많은 비용을 발생하게 합니다.
- ✓ 영속적으로 데이터를 저장하기 위해 객체의 세상과 테이블의 세상을 서로 맞추는 과정이 필요합니다.

객체지향 모델	관계형 모델
객체, 클래스	테이블, 로우
속성(attribute, property)	컬럼
Identity	Primary Key
Relationship/다른 엔티티 참조	Foreign Key
상속/다형성	없음
메소드	SQL 로직, SP, 트리거
코드의 이식성	벤더 종속적임

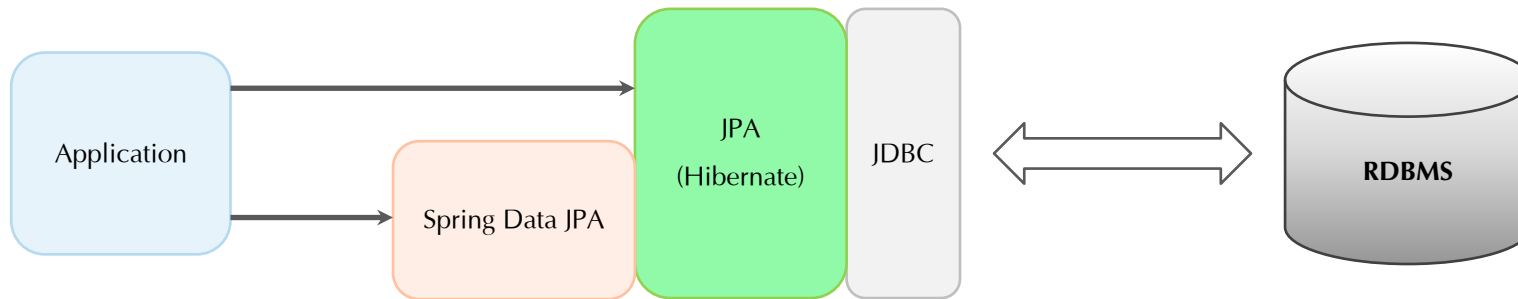
5.2 객체의 세상과 테이블의 세상 (2/2)

- ✓ 자바 어플리케이션에서 관계형 데이터베이스의 사용을 돕는 프레임워크를 **Persistence Framework**라 합니다.
- ✓ **Persistence Framework**는 **SQL Mapping**과 **ORM**으로 구분합니다.
- ✓ **SQL Mapping**은 자바 코드와 **SQL**을 분리하며 개발자가 작성한 **SQL**의 수행 결과를 객체로 매핑합니다.
- ✓ **ORM**은 객체와 관계형 데이터베이스 사이에서 매핑을 담당하며 **SQL**을 생성하여 패러다임의 불일치를 해결합니다.



5.3 Spring Data + JPA의 이해

- ✓ JPA(Java Persistence API)은 자바 프로그램에서 관계형 데이터베이스에 접근하는 방식을 명세화한 인터페이스입니다.
- ✓ JPA는 ORM(Object-Relational Mapping) 기술 표준이며, 구현체로 **Hibernate** 라이브러리를 사용합니다.
- ✓ Spring Data는 영속 데이터 관리를 위한 프로젝트이며 데이터의 저장 기술에 상관없이 다양하게 적용할 수 있습니다.
- ✓ Spring Data + JPA의 의미는 순수 JPA를 통한 데이터 관리에 Spring Data를 더해 좀더 쉽게 영속 데이터를 관리할 수 있도록 하는 것입니다.



5.4 Spring Data JPA 기초 (1/5) – @Entity

- ✓ 도메인 객체를 바로 Entity로 매핑할 수 있지만 임피던스 불일치 등의 이유로 별도의 매핑 클래스(Jpo)를 둡니다.
- ✓ 매핑클래스를 관계형 데이터베이스 테이블로 맵핑할 때 @Entity 어노테이션을 사용합니다.
- ✓ @Entity 어노테이션만 선언했을 때 테이블 이름은 클래스 이름이 되고 대/소문자 치환은 일어나지 않습니다.
- ✓ 대부분은 RDBMS는 대/소문자를 가리지 않으므로 기본JPA 작명규칙이 문제를 발생하지는 않습니다.

```
@Entity
public class TravelClubJpo {

    @Id
    private String id;
    private String name;
    private String intro;
    private long foundationTime;

    public TravelClubJpo(TravelClub travelClub){
        //
        BeanUtils.copyProperties(travelClub, this);
    }
    ...
}
```

5.4 Spring Data JPA 기초 (2/5) – @Table

- ✓ 관습적인 테이블 설계 방법에서 테이블 이름은 대문자에 “_” 를 많이 사용합니다.
- ✓ @Table 어노테이션은 스키마를 지정하거나 테이블 이름을 구체적으로 명시합니다.
- ✓ 특정 RDBMS는 스키마 지정을 테이블 이름에 포함할 수 있기에 “SA.USER” 로 스키마 지정이 가능합니다.
- ✓ @Table(name= “USER” , category= “CATEGORY”)로 카테고리 지정이 가능합니다.

```
@Entity
@Table(name="TRAVEL_CLUB")
public class TravelClubJpo {

    @Id
    private String id;
    private String name;
    private String intro;
    private long foundationTime;

    public TravelClubJpo(TravelClub travelClub){
        //
        BeanUtils.copyProperties(travelClub, this);
    }
    ...
}
```

5.4 Spring Data JPA 기초 (3/5) – @Id

- ✓ JPA에서는 Persistence Provider에게 식별자가 저장된 위치를 알려주는 3가지 어노테이션을 제공합니다.
- ✓ @Id (javax.persistence.Id)
- ✓ @IdClass (javax.persistence.IdClass)
- ✓ @EmbeddedId (javax.persistence.EmbeddedId)

```
@Entity
@Table(name="TRAVEL_CLUB")
public class TravelClubJpo {

    @Id
    private String id;
    private String name;
    private String intro;
    private long foundationTime;

    public TravelClubJpo(TravelClub travelClub){
        //
        BeanUtils.copyProperties(travelClub, this);
    }
    ...
}
```



피해야 할 타입

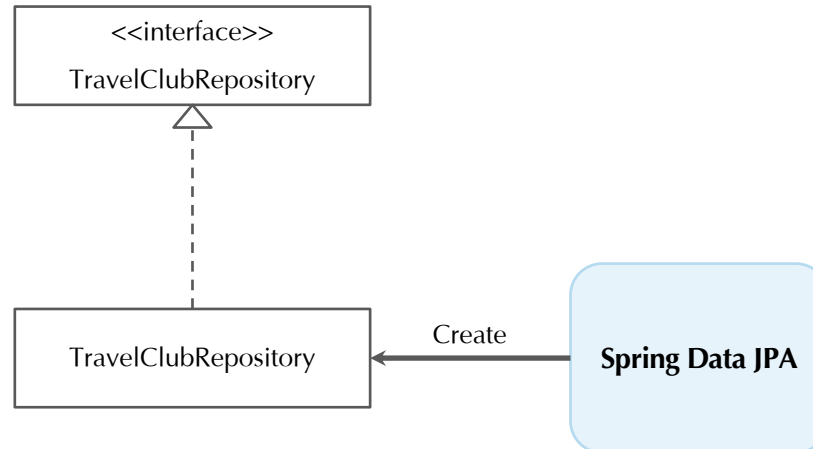
float, Float, double 등을 피해야 합니다

TimeStamp 타입을 피해야 합니다

5.4 Spring Data JPA 기초 (4/5) – JpaRepository

- ✓ Spring Data JPA는 순수 JPA를 이용한 데이터 관리를 보다 쉽게 할 수 있도록 합니다.
- ✓ JpaRepository 인터페이스를 상속하는 것만으로도 기본적인 CRUD 기능을 사용할 수 있습니다.
- ✓ JpaRepository 인터페이스의 제네릭은 <엔티티 타입, 식별자 타입>으로 지정합니다.

```
public interface TravelClubRepository extends JpaRepository<TravelClubJpo, String> {  
    //  
}
```



5.4 Spring Data JPA 기초 (5/5) – Query Methods

- ✓ Spring Data JPA는 Query Methods 기능을 통해 메소드 이름으로 쿼리를 실행할 수 있는 방법을 제공합니다.
- ✓ 순수 JPA에서 EntityManager를 통해 필요한 쿼리를 생성해야 하는 부분을 메소드 이름으로 대체할 수 있습니다.
- ✓ Spring Data JPA는 메소드 이름을 분석해 JPQL 쿼리를 실행합니다.
- ✓ 메소드 이름을 이용한 쿼리 실행은 조회(select)에 해당하며 @Query를 이용하면 다양한 쿼리를 정의할 수 있습니다.

```
public interface TravelClubRepository extends JpaRepository<TravelClubJpo, String> {  
    //  
    List<TravelClubJpo> findAllByName(String name);  
}
```

Keyword	Sample	JPQL snippet
And	findByLastnameAndFirstname	where x.lastname = ?1 and x.firstname = ?2
Or	findByLastnameOrFirstname	where x.lastname = ?1 or x.firstname = ?2
Between	findByStartDateBetween	where x.startDate between ?1 and ?2
Like	findByFirstnameLike	where x.firstname like ?1
StartingWith	findByFirstnameStartingWith	where x.firstname like ?1
Containing	findByFirstnameContaining	where x.firstname like ?1
OrderBy	findByAgeOrderByLastnameDesc	where x.age = ?1 order by x.lastname desc

✓ 토론

감사합니다...

❖ 넥스트리(주)

❖ www.nextree.io